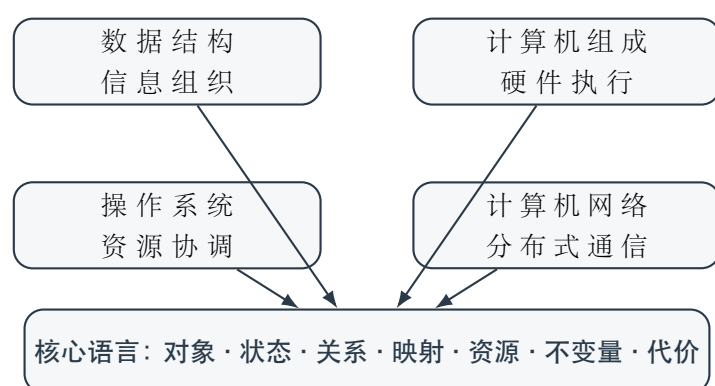


408 四科统一方法论手册

数据结构 · 计算机网络 · 计算机组成原理 · 操作系统

从知识点记忆到问题结构与系统过程推演



使用说明：这不是四本教材的拼接

传统复习往往把四门课拆成大量孤立名词：链表、B 树、TCP、页表、Cache、信号量、DMA、流水线、inode……结果是“知识点都见过，综合题仍然不会推”。这里沿各科自己的核心模型展开机制，再把知识点放回具体问题和动态过程。

跨科观察语言：共同问题不等于共同模型

四门课研究的核心矛盾不同，不能强行套入同一个状态公式。跨科时只共享一组观察顺序：

对象与目标 → 表示或状态 → 操作或事件 → 不变量与边界 → 成本与工作负载

进入具体学科后，立即切换到数据结构、计组、操作系统或网络各自的核心模型。

与其他手册怎样配合

- 《408 统一总图》先建立学科坐标、专题归属和阅读入口；
- 本册展开四科的生成逻辑、关键边界和综合推演；
- 专题手册独立定义具体机制；桥梁手册只解释接口；综合手册只追踪跨机制时间线。

遇到具体算法、协议或内核对象时，应进入对应专题，不在总册之间来回寻找第二份定义。

内容分为两层：

1. **主干层**：理解“为什么必须有这个机制”，训练陌生综合题的推演能力；
 2. **覆盖层**：保留 408 常见知识清单，避免因强调系统结构而漏掉考试细节。
- 对于教材模型与当前工程实现存在差异的内容，统一放入“教材模型 / 工程边界”框；考试按题设作答，工程说明只用于限制绝对化结论。

目录

统一底座：如何理解一个计算系统

1.1 跨科五问的八维展开

五问用于快速进入问题；需要细化时，再展开为以下八个观察维度：

检查维度	需要回答的问题
对象 Object	- 正在被处理的核心实体：元素、节点、边、分组、指令、页、进程、文件、Cache 块 - 实体的身份、生命周期与控制信息
状态 State	- 当前必须记住哪些信息？ - 状态保存在数组位置、节点字段、硬件寄存器、控制块还是协议端点？
作用域 Scope	- 信息在什么范围内有效？ - 单跳 / 端到端 / 一个进程 / 一个地址空间 / 整个机器
映射 Mapping	- 从抽象对象到具体表示经过哪些间接层？ - 下标 → 地址、关键字 → 槽位、逻辑页 → 物理页、域名 → IP 等映射怎样完成？
资源 Resource	哪些操作消耗比较、移动、指针、额外空间、CPU、内存、链路、总线或磁盘 I/O？
事件 Event	- 什么触发状态变化？ - 插入、删除、访问、时钟、中断、ACK、缺页、Cache miss 或系统调用
不变量 Invariant	无论怎样并发调度或突发失败，哪些正确性条件必须一直成立？
代价 Cost	时间、空间、吞吐、等待、带宽、I/O、能耗、公平或可靠性代价是什么？

1.2 四门课各自的核心模型

数据结构

Problem → Relation → Operations → Representation $\xleftrightarrow{\text{Algorithm}}$ Invariant → Cost

研究怎样根据数据关系与操作需求组织信息，使关键操作在维持正确性的同时具有可接受的代价。

计算机网络

Scope → State → Event → Transition → Feedback → Cost

研究多个自治设备如何通过消息交换形成端到端通信。

计算机组成原理

State → Data → Path → Control → Timing → Cost

研究 ISA 的抽象语义怎样成为真实硬件中受时钟约束的数据流动。

操作系统

Resource → Abstraction → State → Event → Mechanism → Policy → Invariant → Cost

研究内核怎样把有限硬件虚拟化并安全地分给多个程序，同时处理并发、I/O 与故障。

1.3 十个反复出现的系统思想

1. 抽象与分层：隐藏下层复杂度，使上层只依赖稳定接口。
2. 间接层与映射：多一次查找，换来重定位、共享、隔离和灵活管理。
3. 时间复用与空间复用：有限资源在不同主体之间分时间或分空间。
4. 机制与策略：能怎样做与应该选谁/选什么严格区分。
5. 局部性与缓存：利用访问可预测性，用小而快的副本隐藏大而慢的资源。
6. 快路径与慢路径：高频正常情况极简，低频异常情况交给复杂处理。
7. 反馈与闭环控制：发送、调度、重传、拥塞等常依赖反馈而不是全局真相。
8. 并发与竞争：多个执行主体同时争用有限资源，正确性和性能都由竞争模式决定。
9. 安全性与恢复：系统不仅要“平时正确”，还要在异常、丢包、崩溃后回到允许状态。
10. 瓶颈与工作负载：不存在脱离 workload 的绝对“最优机制”，优化必须对准瓶颈。

1.4 四道贯穿全课程的综合题

- 数据结构：从操作需求到结构选择；
- 网络：一个 Packet 的一生；

- 计组：一条 **LOAD** 指令的一生；
- OS：一次 `read()` 的一生。

复习时，应不断把新知识挂回这四条推演路径，而不是只放在静态章节目录里。

数据结构：为操作需求设计可维护的表示

2.1 本篇的任务：数据结构真正研究什么

总定义

数据结构不是“容器名称与算法代码的集合”，而是根据问题中的数据关系和操作需求，选择一种可维护的表示，使关键操作在保持正确性的前提下付出可接受的代价。

问题语义 → 逻辑关系 → 操作契约与工作负载 → 存储表示 → 不变量 → 状态转移 → 成本向量

同一批数据可以有多种表示；同一种表示也可能服务不同问题。结构优劣不由名字决定，而由哪些操作多、规模怎样增长、最坏情况能否接受、内存与 I/O 是否受限共同决定。

层次	回答的问题	典型对象
问题与抽象数据类型	使用者需要什么行为，不承诺怎样实现	集合、序列、映射、优先队列、图；查找、插入、删除、取最值
逻辑结构	数据之间是什么关系	线性关系、层次关系、任意网络关系
存储表示	关系怎样编码到内存或外存	连续数组、链式节点、邻接矩阵、邻接表、索引、散列表
算法与不变量	操作怎样改变表示而不破坏结构语义	指针重连、旋转、交换、松弛、合并、失败转移
成本模型	为获得哪些操作能力付出了什么	比较、移动、访存、额外空间、递归深度、磁盘 I/O

五问：整门学科的统一语言

面对任何数据结构问题，依次追问五个问题即可还原整门学科的推理链：

1. 数据是什么关系？——线性、层次还是任意连接？

2. 计算机怎样表示这种关系？——连续存储、链式节点、散列桶还是索引树？

3. 需要频繁做什么操作？——随机访问、查找、插删、取极值还是范围查询？

4. 操作以后什么性质必须继续成立？——有序性、平衡性、偏序、前缀约束还是集合代表元？
5. 为了这些能力付出了什么代价？——时间、空间、比较次数、移动量还是磁盘 I/O？

这五问把看似庞杂的知识点压回一条生成性推理链。以后面对陌生题目，先走一遍五问，再决定用什么结构和算法。

一个数据结构不应被标记为单一复杂度标签，而应理解为一个**成本向量**：

$$C(D) = (C_{\text{search}}, C_{\text{access}}, C_{\text{insert}}, C_{\text{delete}}, C_{\text{traverse}}, C_{\text{space}})$$

结构	随机访问	查找	局部插删	额外空间
顺序表	$O(1)$	无序 $O(n)$	$O(n)$ 移动	小
链表	$O(n)$	$O(n)$	已定位 $O(1)$	指针开销
散列表	不适用	平均近 $O(1)$	平均近 $O(1)$	较高
平衡 BST	按 key $O(\log n)$	$O(\log n)$	$O(\log n)$	指针/平衡信息

因此不能问”数组和链表哪个好”，只能问：**在什么操作集合与频率下，哪个表示更适合？**不存在脱离工作负载的最佳数据结构。

2.2 数据结构的十个核心思想

2.2.1 先看工作负载，再选结构

不存在脱离操作频率的“最好数据结构”。若操作集合为 $O_1, \dots, O_k$ ，频率为 f_i ，可先比较

$$C_{total} \approx \sum_{i=1}^k f_i C(O_i).$$

顺序访问多、随机访问多、动态插入多、范围查询多，会导向不同表示。

2.2.2 逻辑关系不等于存储表示

线性表描述元素的前驱后继关系；顺序表和链表是两种实现。树可以用孩子链表、双亲数组或顺序编号表示；图可以用矩阵或邻接表表示。题目一旦改变表示，操作路径和成本也随之改变。

2.2.3 操作契约决定必须支持的语义

“查找”可能是按下标定位、按关键字判存在、找最小值或做区间查询；“删除”可能已知位置、已知节点指针或只给关键字。先把输入、输出和允许修改的状态说清，才能讨论算法。

2.2.4 不变量是结构的真正定义

数组连续、链表指针闭合、二叉搜索树满足左小右大、堆满足父子偏序、并查集每棵树有代表元。算法不是记忆代码，而是在每次局部修改后保持或恢复这些条件。

2.2.5 局部动作必须解释全局正确性

交换两个元素、重连两条边、旋转一棵子树、松弛一条边都只是局部动作。解题必须回答：动作前哪些区域已经正确，动作后正确区域怎样扩大，未处理区域保留什么性质。

2.2.6 遍历就是维护待处理边界

递归调用栈、显式栈、队列和优先队列都在维护 frontier：哪些状态已经发现但尚未展开。DFS、BFS、树的先中后序和多种图算法，差别首先在 frontier 的取出规则。

2.2.7 用冗余与约束换取速度

排序、索引、平衡信息、散列桶和失败函数都保存了额外结构。它们通过预处理或更新成本减少未来查询成本；更新时则必须支付维护冗余的代价。

2.2.8 时间、空间、预处理与 I/O 是同一组权衡

额外数组可减少重复计算，邻接矩阵用 $O(n^2)$ 空间换常数时间判边，B/B+ 树用高分支降低外存访问层数。复杂度不是单个 $O(\cdot)$ 标签，而是一组与环境相关的成本。

2.2.9 最坏、平均与摊还不能互换

散列查找的平均常数时间不排除最坏线性；动态数组扩容的一次操作可能很贵，但一串追加可摊还为常数；平衡树则主动维护高度，换取可控的最坏界。

2.2.10 边界不是补丁，而是算法骨架的一部分

空结构、单元素、首尾节点、重复关键字、图不连通、溢出和下标越界会改变合法状态集合。若只能在写完主体代码后追加大量特判，通常说明表示或循环不变量没有先说清。

2.3 复杂度：先定义规模与基本操作

写出 $O(n)$ 之前，必须说明 n 是元素数、顶点数、边数、树高、串长还是外存块数，并说明计数的是比较、移动、访存还是 I/O。

复杂度分析固定顺序

1. 定义输入规模及其多个维度，如图的 $|V|$ 与 $|E|$ ；
2. 找循环、递归或数据结构操作中的基本动作；
3. 判断执行次数由规模、数据分布还是结构高度决定；

4. 区分最好、平均、最坏和摊还口径；
5. 再给渐近上界，并补充额外空间或 I/O 成本。

递归算法要同时追踪问题规模怎样缩小与调用栈怎样增长。循环算法要寻找单调进展量，例如区间长度缩小、已排序前缀扩大、未确定顶点数减少。没有进展量，就无法证明终止。

2.4 数据结构的四个统一模型

下面四个模型是整门学科的共同骨架。掌握它们之后，每一种具体结构都只是骨架的不同实例。

2.4.1 模型一：Relation → Representation

第一问永远是逻辑关系是什么，第二问是怎样保存这种关系。典型映射：

线性 → 数组/链表， 层次 → 顺序二叉树/链式树， 图 → 邻接矩阵/邻接表。

这里隐藏着一种普遍 trade-off：**显式表示 vs 隐式表示**。链表用指针真正保存后继关系；数组则通过 $i \rightarrow i + 1$ 由位置计算后继。完全二叉树用 $\text{parent}(i) = \lfloor i/2 \rfloor$ 、 $\text{left}(i) = 2i$ 、 $\text{right}(i) = 2i + 1$ 把父子关系变成下标运算。于是得到第一条方法论：

如果关系能够由位置计算，就不一定需要显式保存关系。

这就是堆特别适合数组、完全二叉树不需要指针的根本原因。

2.4.2 模型二：Operation Set 决定结构价值

结构不是为了”存起来”，而是为了**支持操作**。不同专题实际上在为不同操作集合设计表示：

结构	目标操作集	为此放弃的能力
堆	FindExtreme + Insert + DeleteExtreme	完整有序性；任意 key 查找
并查集	Find + Union	集合内部路径与有序性
散列表	点查找 PointSearch	完整有序；高效范围查询
B+ 树	PointSearch RangeSearch Insert/Delete	+ 简单性；成本模型为 I/O 而非比较 +

堆只维护偏序而非全序，散列表用 Expected $O(1)$ 换掉了有序遍历，并查集的”树”只是实现手段——这些取舍都由目标操作集决定。

2.4.3 模型三：Algorithm = State Transformation

数据结构中的算法不是“一段需要背的代码”，而是**状态转移**：

$$S_t \xrightarrow{\text{Operation}} S_{t+1}$$

例如对 BST 执行 $\text{Insert}(x)$ ，当前 BST_t 变成 BST_{t+1} 。但不是随便一棵树都合法——必须继续满足 $\forall \text{ node} : \text{Left} < \text{node} < \text{Right}$ 。所以算法更准确的定义是：**通过一系列局部修改，把结构从一个合法状态转换到另一个合法状态。**

2.4.4 模型四：Invariant 是数据结构的灵魂

很多学生学 AVL 只记住“LL/RR/LR/RL 四种旋转”，却不知道旋转存在的原因是 BST 有序性 + 平衡不变量 被破坏了。不变量才是结构的真正定义：

- **BST**: $\text{Left} < \text{Root} < \text{Right}$;
- **堆**: $\text{Parent} \leq \text{Child}$ (或 $\geq$);
- **AVL**: BST 有序性 + 任意节点左右子树高度差 ≤ 1 ;
- **红黑树**: BST 有序性 + 着色与黑高约束;
- **B 树**: key 有序 + 分支因子约束 + 叶层等深;
- **并查集**: 每个集合有唯一代表元，父指针最终通向根。

面对陌生算法，**先找 invariant 再理解操作**，往往比逐行模拟代码重要得多。

2.5 贯穿全科的五种设计手段

为什么会有这么多不同的数据结构？不是因为计算机科学家喜欢发明名词，而是它们各自运用了不同的设计手段来优化特定操作。

2.5.1 手段一：利用位置

通过 $\text{index} \rightarrow \text{address}$ 直接推导地址，获得 $O(1)$ 访问。这是最简单的 $\text{LogicalPosition} \rightarrow \text{PhysicalPosition}$ 映射，典型代表是数组。完全二叉树的父子编号也属于这一类。

2.5.2 手段二：增加间接层

通过 $A \rightarrow \text{Pointer/Index} \rightarrow B$ 获得灵活增长、非连续存储和动态更新。典型代表：链表、树指针、散列桶、索引块、邻接表。代价是更多空间、更多查找和 cache locality 可能下降。

2.5.3 手段三：维护额外结构换取未来查询更快

这是整门数据结构极其重要的思想。有序数组提前维护 Order，换取二分查找；BST 维护 LocalOrder；AVL/RB 进一步维护 HeightControl；B/B+ 针对外存维护 HighBranchingFactor；散列表维护 Key $\rightarrow$ Bucket。

查询加速几乎总是来自提前维护的信息。没有免费的快速查找。

2.5.4 手段四：限制操作换取更强语义

栈主动限制”只能在—端操作”，得到 LIFO——天然描述”最近尚未完成的任务”。队列限制”—端进—端出”，得到 FIFO——天然描述”最早等待的任务”。这种”减少自由度换取过程语义”的思想解释了为什么递归、DFS 与栈联系，BFS 与队列联系——这不是巧合。

2.5.5 手段五：利用冗余、局部性和结构性质

对称矩阵 $a_{ij} = a_{ji}$ 不需要存两份；稀疏矩阵只保存非零元素；KMP 利用已匹配前缀中可复用的信息避免完全重来；Huffman 给高频元素更短编码。它们共同体现：

找到输入中的结构性冗余，并把它转化为效率。

2.6 线性结构：相同关系，不同定位与修改代价

表示	优势来源	主要代价	适用信号
顺序表	地址可由基址和下标直接计算；连续存储有利于局部性	中间插删需要移动；容量调整可能搬迁	随机访问、遍历和尾部追加多
链表	已知修改位置时可通过局部重连改变关系	按序号定位需沿链扫描；指针占空间且局部性较弱	结构频繁变化、位置由指针给出

2.6.1 栈与队列：通过限制操作制造时间秩序

栈的后进先出让“最近尚未完成的任务”先恢复，适合递归、括号匹配和深度优先搜索；队列的先进先出让“较早发现的任务”先展开，适合层序遍历和广度优先搜索。循环队列的关键不在取模公式，而在空、满状态怎样区分以及 front/rear 分别指向什么。

2.6.2 数组与压缩：把下标关系变成地址映射

多维数组、对称矩阵和三角矩阵题都应先确定存储顺序，再数目标元素之前有多少个已存元素：

$$\text{Address}(x) = \text{Base} + \text{前置元素个数} \times \text{ElemSize}$$

压缩存储只省略由结构性质可推回的冗余；一旦元素分布不满足假设，映射就失效。

2.7 树：用层次与递归缩小问题范围

树的统一母问题是递归层次结构。树的力量来自两个条件：每个子树仍是同类问题（递归性），且从根到节点的路径编码了层次决策。这种递归定义——Node = Root + LeftSubtree + RightSubtree——直接导致递归算法。树题先画节点身份、父子关系、子树范围和高度，再谈遍历或调整。

2.7.1 遍历：访问时机与 frontier 规则

先序、中序、后序不应背口诀，真正区别是在一次递归访问中，什么时候处理当前节点：先于子树处理为先序，夹在左右子树之间为中序，子树全部完成后为后序。递归实现由调用栈保存未完成现场，非递归实现则显式维护栈。层序遍历使用队列，保证按深度从小到大展开。已知遍历序列重建树时，必须利用能够分割左右子树的序列；仅有先序和后序通常不能唯一确定一般二叉树。

堆的母问题不是 201c 保存完全二叉树 201d，而是 FindExtreme + Insert + DeleteExtreme，因此只维护偏序而非全序。并查集的母问题是动态划分：Find(x)、Union(a, b)，重点不是树本身，而是代表元。Huffman 的母问题是给定权重最小化带权路径长度，树只是优化结果的表示。

2.7.2 不同树由不同不变量生成

结构	核心不变量	获得的能力与代价
二叉搜索树	左子树关键字小于根，右子树大于根（重复键规则需另定）	查找沿一条路径；高度失控时退化
平衡搜索树	在有序性之外约束高度或局部失衡程度	插删后需旋转、变色等修复，换取对数级路径
堆	父节点与孩子满足偏序，完全树保证紧凑表示	常数时间取极值、对数级调整；不支持任意键的有序查找
B/B+ 树	多路有序、节点容量受约束、叶层等高	提高分支因子，减少外存 I/O；更新可能分裂或合并节点
Huffman 树	前缀约束下使带权路径长度最小	高频符号获得短编码；目标不是搜索次序
并查集森林	每个集合由代表元标识，父指针最终通向根	快速合并与判同集；路径压缩和按秩合并优化树高

2.8 图：在任意关系上维护已知区域与待扩展边界

图的根本变化在于每个节点可以与任意多个节点连接，即 $G = (V, E)$ 。图没有天然根和唯一访问顺序。算法必须显式维护：哪些顶点尚未发现、哪些已进入 frontier、哪些结论已经确定，以及跨越已知与未知区域的边。

BFS/DFS 的统一模型

不要只记 201cBFS 用队列，DFS 用栈 201d。真正统一模型是：

Visited + Frontier + ExpansionPolicy

BFS 的 Frontier 是队列，得到按距离层次展开；DFS 的 Frontier 是栈/递归，得到尽量沿一条路径深入。这也是图遍历与树遍历的桥梁。

2.8.1 表示决定可见操作

邻接矩阵用 $O(|V|^2)$ 空间换取常数时间判边，适合稠密图；邻接表把空间和遍历成本压到 $O(|V|+|E|)$ 量级，适合稀疏图。比较算法复杂度前，先确认边在表示中会被扫描多少次，尤其注意无向图的一条边通常存两次。

2.8.2 图算法是在维护不同的不变量

- BFS：队列保证按距离层次展开，首次发现时得到无权图最短边数；
- DFS：栈/递归维护当前探索路径，完成时序可用于连通、拓扑和结构判断；
- Prim：维护已进入生成树的顶点集，以及跨割的最小候选边；
- Kruskal：按边权处理边，并用并查集保证加入边后仍无环；
- Dijkstra：每次确定当前最小暂定距离顶点；存在负权边时该确定性不再成立；
- Floyd：逐步扩大允许作为中间点的集合；
- 拓扑排序：不断删除入度为零的顶点，维护剩余图的先后约束；
- 关键路径：在 DAG 上结合最早与最迟时刻，找出时差为零的关键活动。

最短路算法反复使用松弛：

$$d[v] \leftarrow \min\{d[v], d[u] + w(u, v)\}.$$

公式只是局部更新；真正需要判断的是何时某个 $d[v]$ 已经可以永久确定。

2.9 查找与索引：为减少搜索提前维护信息

查找的根本目标只有一个：**减少候选集**——ReduceCandidateSet。不同结构使用不同的预先信息来排除不可能的候选。

2.9.1 有序性把搜索空间变成可排除区间

折半查找的关键不是背循环代码，而是维护“目标若存在，一定仍在当前候选区间内”。每次比较必须严格缩小区间；判定树则把比较过程转成路径长度，用于分析成功与失败查找成本。

2.9.2 BST $\rightarrow$ 平衡树 $\rightarrow$ B/B+ 树：成本模型驱动结构演进

BST 把有序性转化成动态树结构，但普通 BST 有退化风险（Height = n ）。平衡树引入 PayUpdateCost $\rightarrow$ ControlHeight 的思想，AVL 和红黑树都属此类。当主要成本从 CPU 比较变成存储 I/O 时，目标变成：

$$\boxed{\text{IncreaseFanout} \rightarrow \text{ReduceHeight} \rightarrow \text{ReduceIO}}$$

这就是 B/B+ 树存在的根本原因——非常重要的“成本模型改变 $\rightarrow$ 数据结构改变”实例。

2.9.3 散列把比较问题改成映射与冲突问题

散列完全换思路：Key $\xrightarrow{\text{Hash}}$ CandidateLocation，追求点查找。但冲突是必然的，因为 $|\text{KeySpace}| > |\text{Table}|$ ，所以真正问题变成**如何管理冲突**。散列表先用散列函数把关键字映射到槽位，再处理冲突。装填因子、散列均匀性、开放定址探测规则或链地址链长共同决定实际成本。删除开放定址元素时不能随意置空，否则可能截断其他关键字的探测路径。

2.9.4 KMP 把已匹配前缀压缩成失败状态

朴素匹配在失配后丢弃已知信息；KMP 的核心不是 next 数组公式，而是 Prefix/Suffix Structure $\rightarrow$ Avoid Repeated Comparison。前缀函数或 next 数组记录“当前已匹配后缀还能复用为多长的前缀”。它本质上是字符串匹配状态机的失败转移，不是孤立的下标技巧。不同教材的 next 定义与下标起点可能不同，必须先固定语义再计算。

2.10 排序：通过局部操作逐步制造全局有序

排序的母问题是 $\text{UnorderedState} \xrightarrow{\text{LocalOperations}} \text{OrderedState}$ 。排序算法都在扩大某种已确定区域：插入排序扩大有序前缀，选择排序固定剩余元素中的极值位置，快速排序通过划分确定枢轴两侧关系（一轮后 $\text{Left} \leq \text{Pivot} \leq \text{Right}$ ，这就是它的关键 invariant），归并排序合并两个有序区间，堆排序通过额外维护堆不变量加速反复选择极值。

基数排序甚至不以比较为核心，而利用 key 的数位信息，这提醒我们：Sorting $\neq$ necessarily comparison。

比较排序时至少检查：

1. 当前循环或递归结束后，哪个区间已经满足什么不变量？
2. 主要成本来自比较、交换、移动、递归栈还是额外数组？
3. 最好、平均、最坏复杂度分别是什么，输入初始有序性是否有利？
4. 是否稳定；相等关键字的原相对次序会不会改变？
5. 是内部排序还是外部排序；若数据不入内存，归并路数与初始归并段会怎样影响 I/O 趟数？

稳定性判断

稳定性是算法对相等关键字相对次序的保证，不是某一次样例中“看起来没变”。判断时要检查算法是否可能让相等元素跨越，或交换到彼此两侧。

如果数据远大于内存，真正昂贵的是外部 I/O，算法目标改为 **Minimize Number of Passes**——这就是初始归并段、多路归并、败者树和置换选择存在的真正原因。

2.11 七条深层规律

以下七条规律贯穿所有数据结构专题，是比任何单独算法都更稳定的知识。

2.11.1 表示决定可访问信息

数组直接暴露 Position，链表暴露 Neighbor，BST 暴露 RelativeOrder，堆暴露 Extreme，散列暴露 CandidateBucket。

数据结构的能力，本质上来自它主动保存了哪些额外信息。

2.11.2 快速查询通常意味着付费维护

如果希望未来查找更快，往往需要提前维护有序性、索引、平衡、散列映射或堆性质。 $C_{\text{Query}} \downarrow$ 几乎总是伴随 $C_{\text{Update/Space}} \uparrow$ 。

2.11.3 完整信息并不总值得维护

堆是绝佳例子：只需 FindMax，没有必要维护完全有序，于是只维护 $\text{Parent} \geq \text{Child}$ 。数据结构设计不是保存越多关系越好，而是保存足够完成目标的最少信息。

2.11.4 结构不变量比实现步骤更稳定

代码可能有几十种写法，但 BST invariant、Heap invariant、AVL balance invariant、Union-Find partition invariant 不变。面对陌生代码题，先找 invariant 往往比逐行模拟更重要。

2.11.5 遍历 = 受控探索

树、图、搜索问题都可以抽象为 Current + Frontier + Visited/Completed。决定“下一个扩展谁”就是 traversal strategy。

2.11.6 有序性是信息

无序集合查找约 $O(n)$ ，有序数组查找 $O(\log n)$ ，BST 利用局部有序性，B+ 利用有序加索引层次。“有序”不是为了好看，它是一种可以用来排除候选空间的信息。

2.11.7 成本模型改变，最佳结构就可能改变

RAM 中指针遍历或许可以，磁盘上一次随机 I/O 代价巨大。普通 BST 高度约 $\log_2 n$ 都可能太高，B 树高度约 $\log_m n$ ($m \gg 2$)，大幅降低 I/O。

数据结构的选择取决于底层机器的成本模型。

这也是数据结构与计组、操作系统最重要的接口之一。

2.12 综合题：从操作需求到结构选择

面对“该选什么结构”，固定按以下路径推演：

数据关系 → 操作及频率 → 正确性要求 → 候选表示 → 成本比较 → 选择

需求信号	优先候选	必须继续核对的代价
频繁按下标访问、顺序扫描	顺序表/数组	中间插删与扩容成本
已知节点位置后频繁局部插删	链式结构	定位成本、指针空间与局部性
反复取得当前最大或最小元素	堆/优先队列	任意查找与删除是否也需要高效
动态判定关键字是否存在	散列表或平衡搜索树	是否需要有序遍历、范围查询与最坏界
频繁合并集合并判断连通	并查集	是否还需要输出集合内部路径或边
外存上的大规模有序索引	B/B+ 树	节点容量、树高与块 I/O

这张表只生成候选，不直接给答案。题目一旦加入“必须稳定”“内存受限”“要求最坏 $O(\log n)$ ”“需要范围查询”等约束，选择就可能改变。

设计者视角：四个典型选择情境

- 1. 频繁 Insert 和 DeleteMax，不关心完整顺序 → **堆**；
 - 2. 频繁点查找，不要求顺序遍历 → **散列表**；
 - 3. 既要点查找又要范围查询 → 散列失去优势，考虑**有序索引**（平衡 BST / B+ 树）；
 - 4. 数据在外存，随机 I/O 昂贵，成本模型从 Comparison 变成 IO → BST 不够，需要**B/B+ 树**。
- 这就是数据结构真正的设计者视角：**需求** → **操作集** → **候选结构** → **trade-off** → **选择**。

2.13 408 数据结构解题检查表

408 训练：数据结构做题十问

- 1. **Goal**: 题目到底要求什么？
- 2. **Objects**: 操作对象是什么，之间是线性、层次、网络关系还是集合/映射？
- 3. **Representation**: 题目采用什么表示，每个下标、指针、字段或矩阵位置表示什么？
- 4. **Invariant**: 当前必须保持什么不变量？
- 5. **Operation**: 本步骤到底改变什么？输入、输出和允许修改的状态是什么？
- 6. **State**: 执行以后哪些位置/指针/集合状态改变？哪个区域已经确定？
- 7. **Progress**: 每一步怎样推进，为什么一定终止且不会漏解或重复处理？
- 8. **Boundary**: 空结构、首尾、单节点、重复关键字、不连通和越界怎样处理？
- 9. **Cost**: 时间、空间、比较、移动或 I/O 成本是什么？规模与基本操作是什么？

10. **Verify**: 结果是否仍满足 invariant? 最坏/平均/摊还口径是否匹配?

数据结构高频混淆

逻辑结构 $\neq$ 存储表示; 顺序存储 $\neq$ 逻辑有序; 已知节点插删快 $\neq$ 找到节点也快; 堆有序 $\neq$ 全序; 平均 $O(1)$ $\neq$ 最坏 $O(1)$; 树高 $\neq$ 节点数; 连通 $\neq$ 路径唯一; 最短路 $\neq$ 最小生成树; 稳定 $\neq$ 原地; 算法跑出样例 $\neq$ 不变量已证明。

数据结构七个典型错误模式

1. **混淆逻辑结构与存储表示**: “树就是链式结构”是错误的——树可以链式、顺序、双亲数组或孩子兄弟表示。
2. **只背代码不知 invariant**: 只会背 AVL 四种旋转, 题目稍改就不会——真正问题是平衡不变量被破坏。
3. **脱离 workload 比较结构**: “Hash 和 BST 谁好?” 是错误问题——必须先问点查找、范围查询、更新、有序输出的比例。
4. **把复杂度当代码行数**: 复杂度研究的是 $\text{InputSize} \rightarrow \text{BasicOperationCount}$, 不是程序有几层 if。
5. **忘记初始条件改变复杂度**: 树高、数组有序性、图密度、装填因子、pivot 质量都会改变成本。
6. **模拟算法却不记录已确定区域**: Quick Sort 一轮后 pivot 已在最终位置, Selection 某个极值已确定——应不断问“现在哪些部分已不需要再怀疑”。
7. **用主观标签代替可观察行为**: 说“粗心”“基础差”不是诊断——应具体指出在哪一步、属于概念/公式/推理/计算的哪类错误。

2.14 数据结构与其他三科的接口

2.14.1 数据结构 $\times$ 计算机组成原理

最重要接口是 $\text{LogicalRepresentation} \rightarrow \text{MemoryLayout} \rightarrow \text{ActualAccessCost}$ 。数组连续存储带来空间局部性 (Spatial Locality), 链表指针跳转逻辑复杂度可能仍是 $O(n)$ 但 cache 行为可能完全不同。因此 $\text{AsymptoticComplexity} \neq \text{ActualMachineCost}$ 。

2.14.2 数据结构 $\times$ 操作系统

OS 内部大量使用数据结构: 就绪队列、等待队列、页表结构、文件系统索引、环形缓冲区等。数据结构负责解释“结构为什么具备某种操作成本”, OS 负责解释“为什么这个子系统需要这种操作集”。

2.14.3 数据结构 $\times$ 网络

图 $G = (V, E)$ 天然成为路由、拓扑和最短路径的抽象基础。队列天然连接分组缓冲与调度。

2.15 专题导航索引

整个数据结构体系建议组织为以下专题，每个专题对应一份专题手册：

编号	专题	核心问题
DS-00	数据结构学科总图	Relation + Representation + Operation + Invariant + Cost
DS-01	算法复杂度与成本模型	InputSize → OperationCount → AsymptoticCost
DS-02	线性表	Contiguous vs Linked
DS-03	栈与队列	RestrictedOperations → LIFO/FIFO
DS-04	数组与特殊矩阵	IndexMapping + Compression
DS-05	树与二叉树	Hierarchy + RecursiveTraversal
DS-06	Huffman、堆、并查集	Coding, Priority, DynamicPartition
DS-07	图表示与遍历	Representation + Frontier
DS-08	图算法	MST, Relaxation, PartialOrder, CriticalPath
DS-09	静态查找	Structure → CandidateReduction
DS-10	动态有序索引	BST → AVL/RB → B/B+
DS-11	散列	Key → Bucket + CollisionManagement
DS-12	KMP	ReuseMatchedInformation
DS-13	内部排序	LocalOrdering → GlobalOrdering
DS-14	外部排序	MinimizeExternalIO

三个桥梁专题

- **Bridge A：遍历统一模型**——连接递归、栈、队列、树遍历、DFS、BFS，统一为 Frontier + Visited + ExpansionOrder；
- **Bridge B：有序性、索引与查询成本**——统一比较 SortedArray、BST、AVL/RB、B/B+、Hash，核心是“为了让查询快，到底维护了什么额外信息，Update/Space/RangeQuery 付出什么代价”；
- **Bridge C：数据结构作为算法的加速部件**——Kruskal = Sorting + UnionFind, Prim/Dijkstra = Graph + PriorityQueue, BFS = Graph + Queue, DFS = Graph + Stack。复杂算法往往不是神秘新结构，而是若干简单操作需求组合后调用合适的数据结构作为子程序。

2.16 数据结构一页压缩版

如果考前只能留下一页，请留下以下内容。

总定义

数据结构是在特定成本模型下，为一组操作需求选择并维护合适的信息组织方式。算法则是让这种组织方式在不断变化的数据上仍然保持合法，并完成目标操作。

总推理链：

Problem $\rightarrow$ Relation $\rightarrow$ Operations $\rightarrow$ Representation $\rightarrow$ Invariant $\rightarrow$ Algorithm $\rightarrow$ Cost

三个核心区分：

LogicalStructure $\neq$ StorageRepresentation, Structure $\neq$ Algorithm, AsymptoticCost $\neq$ ActualMachineCost.

四个核心思想：

1. 表示决定操作成本；
2. 不存在脱离 workload 的最佳结构；
3. 算法是在修改结构的同时维护 invariant；
4. 更快的查询通常来自提前维护更多结构信息。

做题十问：Goal $\rightarrow$ Objects $\rightarrow$ Representation $\rightarrow$ Invariant $\rightarrow$ Operation $\rightarrow$ State $\rightarrow$ Progress $\rightarrow$ Boundary $\rightarrow$ Cost $\rightarrow$ Verify。

计算机网络：在没有全局即时状态的世界中协作

3.1 本篇的任务：建立网络世界的坐标系

本篇不是计算机网络教材的缩写，也不负责完整讲解 TCP、IP、CRC、OSPF、DNS 等具体协议。这些机制将在后续专题中分别打穿。本篇只负责三件事：

- 回答计算机网络为什么必然需要这些机制；
- 建立观察任意网络问题时都能复用的统一坐标系；
- 规定后续各专题分别负责什么，使它们彼此独立而不重复。

因此，本篇拥有的是一套观察任意网络问题的统一坐标系：

本篇的总框架坐标系

Scope + State Ownership + Transition + Feedback + Resource/Cost

本篇只负责建立这一坐标系并界定后续专题边界，而不是讲解具体协议实现。

以后看到任何陌生网络机制，首先不要问：“它属于哪一层？”而要先问：它在什么作用域内解决谁的什么问题？谁保存了完成这个决策所需的状态？什么事件让这些状态发生变化？

3.2 网络究竟是什么？

3.2.1 网络不是一根很长的管道

如果两台计算机只是通过一根无限快、绝不出错、永不拥塞的线连接起来，那么很多网络协议根本没有存在的必要。现实世界恰恰相反。计算机网络由大量彼此自治的设备组成：

- 主机只知道自己的局部状态；
- 交换机只掌握局部链路信息；
- 路由器只能依据当前已有的转发状态做决定；
- 远端状态不能被瞬间观察；
- 信息只能通过报文传播；
- 信息传播本身还需要时间。

计算机网络的本质定义与核心难题

计算机网络是大量自治节点依据局部状态，对收到的消息做局部决策，最终共同形成端到端通信的分布式系统。

这也决定了网络最核心的困难不是“数据怎么在线里跑”，而是：

没有全局即时状态时，多个节点怎样通过消息协作？

旧版总图已经抓住了这一点，并从六个基本矛盾解释互联网为什么会形成今天的结构。

3.3 六个不可消除的约束：网络机制从哪里长出来？

几乎整个 408 网络都可以从六个现实约束生成。

根本约束	直接后果	被迫出现的机制
距离 Distance	信息传播需要时间	时延、RTT、Timer、Pipeline、Window
异构 Heterogeneity	不同链路和设备无法直接统一	Layering、IP、统一网络层抽象
共享 Sharing	多个通信流争夺同一资源	Multiplexing、Queue、MAC、Congestion Control
不可靠 Unreliability	数据可能错、丢、重复、乱序	Error Detection、ACK、Seq、Retransmission
规模 Scale	无法为全球每个节点保存完全独立状态	Hierarchy、Prefix、CIDR、Aggregation、AS
无全局即时状态 No Global Instantaneous State	节点只能依据延迟到达的局部知识判断	Distributed Routing、Feedback、Timeout、Convergence

这里最重要的不是背右侧机制，而是掌握：

Constraint → Failure → Mechanism

例如：如果不存在丢包：ACK, Timeout, Retransmission 就没有必要。如果不存在传播时延：Pipeline, Window 的重要性会大幅下降。如果链路资源无限：Queue, Congestion 也不会成为核心问题。

所以学习一个协议时始终追问：它到底是被哪一个不可消除的现实约束逼出来的？

3.4 第一坐标轴：Scope——一个决策到底负责多远？

网络最容易犯的一类错误，是把不同作用域里的对象混在一起。因此网络分析的第一坐标不是“协议层名称”，而是 Scope（作用域）。五层模型真正有价值的地方，是它把不同尺度的通信责任分开。

3.4.1 五种作用域

- **Application：语义**

回答：双方究竟在交换什么？

例如：HTTP Request / Response；DNS Query / Answer；SMTP Message。

核心不是“怎样运输”，而是：**Meaning**

- **Transport：端点**

回答：到达主机以后，应该交给哪个通信端点？两端之间需要怎样的通信条件？

出现：Port；Connection；Seq；ACK；Flow Control；Reliability。

核心是：**Process/Endpoint → Process/Endpoint**

- **Network：跨网络路径**

回答：目的地在哪里？当前节点接下来往哪走？

出现：IP Address；Prefix；Forwarding；Routing。

核心是：**Host/Network → Host/Network**

- **Link：当前一跳**

回答：在这一条具体链路上，这一帧怎样送给下一设备？

出现：Frame；MAC Address；CRC；Ethernet；Wi-Fi；Switch。

核心是：**One Hop**

- **Physical：一段介质**

回答：0 和 1 怎样真正变成可传播的物理信号？

出现：Symbol；Encoding；Modulation；Bandwidth；Signal。

核心是：**Signal**

因此可以把整个纵向通信链压成：

纵向通信主链：语义 → 端点 → 路径 → 下一跳 → 信号

旧版已经建立了这个作用域模型。

3.5 封装到底是什么？

“每层加一个首部”只是表面现象。更本质地说：每向下一作用域移动一步，就必须加入这一作用域完成局部决策所需的信息。

因此：*ApplicationMessage* 关心：What does it mean? Transport 增加：*Port, Seq, ACK, ...* 因为端点需要做端到端决策。Network 增加：*IP, TTL, ...* 因为路由器需要决定路径。Link 增加：*MAC, FCS, ...* 因为当前链路需要决定一跳交付。

所以：

Message → Segment → Datagram → Frame → Signal

不是简单地：“越来越多层头部包在外面”，而是：**不同作用域的控制信息在逐层叠加**。接收方进行解封装，则是在逐层完成对应作用域的决策以后，剥离已经失去作用的信息。

3.6 第二坐标轴：State——谁必须知道什么？

仅知道作用域还不够。网络中的每个决定都依赖某种状态。因此第二个固定问题是 **State Ownership**（状态所有权），也就是：为了做这个决策，谁必须保存什么信息？

3.6.1 TCP 状态在哪里？

TCP 的 Connection State、Sequence Number、ACK State、Receive Window 主要存在于 **Endpoints**（通信端点）。普通 IP 路由器通常不需要知道某条 TCP 连接当前序号是多少。

3.6.2 交换状态在哪里？

交换机保存 MAC → Port 映射。它只需要回答：当前这一帧应该从哪个端口发出去？

3.6.3 路由状态在哪里？

路由器保存 Prefix → NextHop/Interface 映射。它需要回答：当前 IP 分组下一步去哪？

3.6.4 DNS 状态在哪里？

Resolver 可以保存 Name → ResourceRecord 以及 TTL 约束下的缓存。

状态保存重要原则

协议存在 $\neq$ 所有网络设备都保存该协议状态

旧版已经明确区分了 TCP endpoint state、switch table、router forwarding state 与 DNS cache。

3.7 网络的统一动态模型

有了 Scope 和 State，才真正可以描述网络怎样运行。

网络的统一动态模型

网络协议的核心不是静态报文格式，而是**局部状态机的动态演化**：

Local State + Message/Timer $\rightarrow$ Action + New State + Feedback

更正式地：

$$S_i \xrightarrow{M/T} S'_i$$

其中 S_i 为节点当前局部状态， M 为收到的 Message， T 为 Timer expiration 等本地事件， S'_i 为处理后的新状态。节点随后可能产生新消息 M' 改变另一个节点的状态。

因此整个网络不是一个拥有全局控制器的大状态机，而是**大量局部状态机通过消息相互耦合的分布式系统**。

3.8 为什么 Timer 在网络中如此重要？

本机无法直接读取远端内存。发送方不能瞬间知道：“对方到底收到了没有？”只能通过 *ACK* 获得正反馈。如果什么都没收到，就必须利用 **Time** 推断某些事情可能出了问题。于是产生：*Timeout* → *Retransmission*。

因此在网络里，**Message + Timer** 是驱动状态机最基本的两类输入。这也是为什么 TCP、可靠传输、DHCP、ARP、路由收敛等看似完全不同的机制，都可以用同一种状态机语言推演。TCP 标准本身就是围绕连接状态、序号空间和收到不同 segment 后的状态变化组织的。

3.9 三条必须贯穿所有专题的关系链

Scope 与 State 是坐标。下面三条链则负责把大量具体协议连接起来。

3.9.1 Naming → Addressing → Forwarding → Delivery

用户通常不会从：“我要访问 142.250.x.x”开始，而是从一个名字开始：**Name**（例如域名）。

第一步：*Name* → *Address*

随后：*DestinationIP* → *ForwardingDecision* → *NextHop*

真正进入当前链路以后还需要：*NextHopIP* → *LinkAddress*

因此形成完整寻址链：

Name → Address → Route/Forwarding → NextHop → LinkAddress

在经典 IPv4 局域网模型中，可以具体理解为：*Domain* → *IP* → *NextHopIP* → *MAC*。旧版已经建立了这条 *Name* → *Address* → *Route* → *Next-hop Link Address* 链。这条链最重要的作用，是阻止学生把 Domain, IP, MAC, Port 看成“不同格式的地址”。它们实际上属于不同作用域、解决不同问题。

3.10 Forwarding 与 Routing 必须彻底分开

这是整个网络层最重要的边界之一。

核心概念区分：Forwarding 与 Routing

Forwarding（转发） 局部的 Data-Plane 决策：这个 packet 到了当前路由器，现在往哪发？

DestinationIP $\xrightarrow{\text{LongestPrefixMatch}}$ NextHop

Routing（路由） 全局的 Control-Plane 协作：Forwarding Table 是怎么形成的？

LocalKnowledge + RoutingMessages → UpdatedRoutingState

Control Plane → Produces/Updates Rules Data Plane → Applies Rules

旧版已经建立了 Data Plane / Control Plane 的区分；专题体系中要进一步把二者交给不同 canonical owner。

3.11 可靠性不是某一层天然拥有的能力

可靠传输母模型

Reliability is a requirement, not a layer. 一条链路可以做 Hop-Level 可靠性，端系统也可以做 End-to-End 可靠性。二者并不重复，因为 Scope 不同。

真正统一的可靠传输母模型是：

Sequence + ACK + Timer + Retransmission + Window

为什么需要这些机制？

- 数据可能损坏 → 需要 ErrorDetection。
- 数据可能丢失 → 需要 ACK+Timeout+Retransmission。
- ACK 也可能丢失 → 需要区分 NewData 和 Duplicate → 于是出现 SequenceNumber。
- Stop-and-Wait 利用率太低 → 允许 MultiplePacketsInFlight → 产生 Window。

因此 Stop-and-Wait、GBN、SR 和 TCP 中的大量可靠机制应共享一个“可靠传输”母模型，而不是彼此孤立记忆。

3.12 Reliability、Flow Control、Congestion Control 必须三分

三个机制都可能改变“发送多少”，因此特别容易混。但它们保护的对象完全不同。

问题	保护对象	核心问题
Reliability	通信语义	数据有没有正确到达？
Flow Control	Receiver	接收方吃不吃得下？
Congestion Control	Network	网络路径扛不扛得住？

因此：

Reliability ≠ FlowControl ≠ CongestionControl

TCP 中发送能力通常同时受到接收窗口和拥塞窗口约束：

$$W_{\text{send}} \leq \min(rwnd, cwnd)$$

旧版已经建立了这一三分结构。以后不再把它作为一个孤立“易错点”，而是理解为：不同状态所有者，对发送行为施加了不同约束。

3.13 网络为什么会拥塞？

3.13.1 统计复用的收益

分组交换允许多个通信流按需共享链路。在低负载时能够大幅提高资源利用率 (Resource Utilization)，避免提前为每一个用户永久预留最大带宽。

3.13.2 统计复用的代价

当 $ArrivalRate > ServiceRate$ 持续存在时，队列增长导致 $QueueLength \uparrow \rightarrow QueueingDelay \uparrow \rightarrow BufferFull \rightarrow Loss$ 。如果丢包又引起更多重传 ($Loss \rightarrow Retransmission \rightarrow MoreLoad$)，系统可能进一步恶化。

拥塞的结构性本质

Statistical Multiplexing $\rightarrow$ Queue $\rightarrow$ Congestion Risk

拥塞不是 TCP “制造”的问题，而是**有限共享资源**本身带来的结构性必然。TCP 拥塞控制只是尝试利用反馈动态调节发送速率。

3.14 Congestion 的母模型：反馈控制

拥塞控制的母模型：反馈控制

OfferedLoad $\rightarrow$ NetworkState $\rightarrow$ CongestionSignal $\rightarrow$ SenderAdjustment

拥塞控制的真正主题不是死记硬背慢启动公式，而是：

端点看不到网络内部完整状态时，怎样依据有限反馈推断网络还能承受多少负载？

这再次印证了网络的根本现实——**No Global Instantaneous State**。因此 *cwnd* 本质上是发送方对网络容量的局部状态估计 (Sender's estimate/control state about network capacity)。

3.15 网络性能必须用“时间—在途数据—瓶颈”理解

3.15.1 发送时延与传播时延不是一回事

发送时延 (d_{trans}) $d_{trans} = \frac{L}{R}$ ，回答：把 L bit 数据推入速率为 R 的链路需要多久。

传播时延 (d_{prop}) $d_{prop} = \frac{D}{v}$ ，回答：信号在距离为 D 、传播速度为 v 的介质中传输需要多久。

记住绝对区别：**Transmission $\neq$ Propagation**。

3.16 BDP：为什么窗口必须和路径尺度匹配？

带宽时延积 (Bandwidth-Delay Product)：

$$BDP \approx Bandwidth \times Delay$$

这代表了路径上能够容纳的最大“在途数据量”。如果 $Window \ll BDP$ ，即使链路带宽极高，发送方也会陷入“发一点 $\rightarrow$ 等待 ACK $\rightarrow$ 再发一点”的饥饿状态。因此，滑动窗口的本质是**控制在途数据量 (Control Amount of Data in Flight)**，使其匹配路径的 **Bandwidth-Delay Scale**。

3.17 Bottleneck：优化非瓶颈为什么可能没有用？

端到端路径的长期吞吐量受到最窄资源的瓶颈限制：

$$Throughput \lesssim \min(R_1, R_2, \dots)$$

提高非瓶颈链路的速率无法提升全局吞吐量。这一思想贯穿 Router Queue、Receiver Processing、TCP Window 与 Disk/Network I/O。网络性能分析的第一步永远是：先找 Bottleneck。

3.18 分层的本质：隔离变化，而不是分类知识点

Layering 最大的价值是 Isolate Change（隔离变化）。通过建立 Stable Interface，使得底层的介质更换（如光纤换 Wi-Fi）不影响上层的语义表达（如 HTTP）。必须记住：Layering = Analysis/Architecture Principle，而不是自然界不可跨越的物理定律。现代系统中 NAT、VPN、Proxy、QUIC 等都会以不同方式跨越传统层次边界。

3.19 八个核心专题：后续知识应该由谁拥有？

总图只建立问题。具体机制必须有唯一 canonical owner。

编号	专题名称与母问题	Canonical Owner / 核心母模型
NW-01	通信基础与网络性能 一个 bit 穿过真实信道付出什么代价？	Bit/Symbol, Encoding, Shannon, Transmission vs Propagation Signal → Bit → Link → Delay/Capacity
NW-02	单跳交付 (MAC / LAN) 确定下一设备后怎样把数据交过去？	Framing, CRC, Shared Medium, CSMA, Ethernet, Switch Bits → Frame → MediumAccess → OneHopDelivery
NW-03	可靠传输模型 怎样在不可靠信道上构造可靠服务？	Stop-and-Wait, Seq, ACK, Timeout, Retransmission, Window Seq + ACK + Timer + Retransmission + Window
NW-04	IP 地址与分组转发 分组到达节点后下一跳去哪？	IPv4/v6, Subnet, CIDR, LPM, Forwarding, Gateway, ARP, DHCP, NAT DestinationAddress → PrefixMatch → NextHop
NW-05	路由 (分布式知识) Forwarding Table 从哪里来？	Distance Vector, Link State, RIP, OSPF, BGP LocalKnowledge + Messages → DistributedConvergence

NW-06	运输层 (UDP / TCP) IP 到主机后怎样构造进程 间通信?	Port, Multiplexing, UDP, TCP Connection, Handshake, TIME_WAIT Host → Endpoint → ConnectionState
NW-07	拥塞 (反馈控制) 需求超过网络能力时怎样 避免压垮?	Queue growth, Congestion Signal, cwnd, AIMD, Slow Start Load → Feedback → RateAdjustment
NW-08	应用层 (发现与语义) 底层传送 bytes 后应用定 义什么?	Discovery (Name → Address), Semantics (Bytes → Meaning), Interaction Name → Address / Bytes → Meaning

3.20 一个 Packet 的一生：整个专题体系的导航器

这条链负责告诉学生：八本专题在真实通信中在哪里接起来。假设一台刚接入网络的主机第一次访问某个远程网站。

- Phase 1: 网络身份 通过 DHCP 获得 IP+Mask+Gateway+DNS (关联 NW-04)。
- Phase 2: 名称发现 通过 DNS 将 Domain Name 变为 IP (关联 NW-08)。
- Phase 3: 跨网决策 判断 DestIP 是同一子网还是需要走 DefaultGateway (关联 NW-04)。
- Phase 4: 寻址下一跳 使用 ARP 将 NextHopIP 解析为 NextHopMAC (关联 NW-04/02)。
- Phase 5: 单跳交付 构造 Frame，交换机依据 MAC 表完成局域网内转发 (关联 NW-02)。
- Phase 6: 路由与转发 沿途 Router 解帧、TTL-、LPM、重新封装 (关联 NW-04/05)。
- Phase 7: 端点连接 IP 送达后，通过 Port 交付给进程，完成 TCP 握手与数据传输 (关联 NW-06/07)。
- Phase 8: 语义解释 应用层解析收到的字节流 (关联 NW-08)。

因此一条完整主链可表达为：

Name → IP → Route → NextHop → MAC → Frame → Router → Endpoint → Application

3.21 解综合题：同时追踪四条流

解综合题：同时追踪四条流

- Scope Flow Application → Endpoint → EndToEndPath → OneHop → PhysicalMedium
- Name / Address Flow Domain → IP → Prefix → Port → MAC (各不相同，不可混淆)
- State Flow DNS Cache → ARP Table → Switch Table → Forwarding Table → TCP State
- Message / Feedback Flow Query/Reply → SYN/ACK → Data/ACK → RoutingUpdate → ICMP

这四条线同时跟踪，绝大多数网络综合题会突然变得清楚。

3.22 网络最重要的概念边界

这些不是零碎“易错点”，而是作用域和状态所有权不同产生的结构性边界。必须严格区分：

概念对比与非等价关系	本质原因与作用域差异
Name ≠ IP ≠ MAC ≠ Port	属于应用层语义、全局网络拓扑、局域网单跳身份与主机进程，作用域完全不同。
Routing ≠ Forwarding	路由是控制平面的全局分布式收敛；转发是数据平面的单点硬件级查表。
Reliability FlowControl CongestionControl	≠ 保护通信内容 vs 保护接收方缓冲区 vs 保护网络路径容量。
TransmissionDelay PropagationDelay	≠ 把数据推进链路的时间 (L/R) vs 信号在介质上传输的时间 (D/v)。
Connection PhysicalPath	≠ 端系统抽象出来的逻辑状态 vs 沿途实际经过的物理链路与路由器。
LinkReliability EndToEndReliability	≠ 局部单跳链路防错 vs 端到端最终可靠性（Scope 不同）。
TCPState RouterState	≠ 存在于 Endpoints 的连接状态 vs 存在于 Router 的转发/路由表。
Layering ≠ PhysicalLaw	分层是架构与分析原则（Isolate Change），不是物理客观定律。

3.23 网络统一七问

408 训练：网络七问

1. **Scope**：当前问题发生在哪个作用域？
2. **Object**：当前正在处理什么对象（Message / Segment / Datagram / Frame / Bit）？
3. **State Owner**：做这个决定所需的信息保存在哪里？
4. **Event**：刚刚发生了什么（Message arrival, timeout, collision）？
5. **Transition**：哪个状态、字段、映射或队列因此改变（ $S_t \rightarrow S_{t+1}$ ）？
6. **Feedback**：当前结果怎样影响别的节点？
7. **Target / Cost**：题目最后到底问什么？并检查 Unit, Range, Bottleneck, Scope。

3.24 总手册与专题手册的职责边界

网络总图不保存完整的五层知识点列表。本篇只拥有：**DistributedCommunication**、**Scope**、**StateOwnership**、**State** ^{Message/Timer}→ **Transition** → **Feedback**、**StatisticalMultiplexing** → **Queue** → **Congestion**。本篇负责 **Map**，专题负责 **Depth**，桥梁专题负责 **Interface**，综合专题负责 **Integration**。四者不再争夺相同知识。

3.25 408 四科核心概念边界与反混淆对齐

概念 A	概念 B	真正区别	题目信号	混淆后果
Ready (就绪)	≠ Blocked (阻塞)	仅缺 CPU 时间片 vs 等待外部 I/O 或锁资源	“等待事件到来”/“获得时间片”	把等待锁写成 Ready
Mode Switch	≠ Context Switch	仅特权级切换 (不换 PCB) vs 寄存器与页表上下文全换	中断响应/系统调用 vs 进程调度	误算上下文保存代价
进程	≠ 程序	动态执行实体与资源容器 vs 静态磁盘代码二进制	“可执行文件”/“PCB 创建”	把程序属性套在进程上
线程共享	≠ 全部状态共享	共享全局与堆内存 vs 独立栈帧与寄存器集合	“局部变量”/“栈溢出”	误认为线程栈私有可被直接覆盖
Mutex/Lock	≠ Condition Var	互斥资源锁 vs 事件等待与唤醒协调	pthread_mutex vs pthread_cond	乱用锁代替条件变量
Signal	≠ Condition True	信号仅代表提醒 vs 条件可能仍被其他线程抢占	“while 循环判条件”	改用 if 导致虚假唤醒崩溃
Unsafe State	≠ Deadlocked	存在死锁风险路径 vs 实际已陷入循环等待不可推进	银行家算法/安全序列	把不安全状态判为已死锁
Virtual Addr	≠ Physical Addr	逻辑 CPU 视角的连续空间 vs 物理 DRAM 块的散布页框	MMU/页表/段页式	把 VA 当实际物理存储
TLB Miss	≠ Page Fault	TLB 缓存缺失找页表 vs 页面不在内存需磁盘 I/O	内存访问时间/缺页中断	混淆微秒级与毫秒级延迟
Buffer	≠ Cache	缓解读写速度不匹配的缓冲 vs 缓存热点数据避免重复访存	I/O 搬运 vs CPU 访存	混淆物理职责

Write 返回	≠ 稳定入盘	数据仅写至内核页缓存 vs 数据物理持久化至磁盘	fsync() / 崩溃一致性	误判数据可靠性
文件名	≠ inode	目录项中的名称映射 vs 文件的物理元信息与数据指针	dentry/软硬链接	把文件名当物理文件实体

3.26 最终压缩：网络真正应该留下什么？

计算机网络的终极总结

留下核心主线：

Scope → State $\xrightarrow{\text{Message/Timer}}$ Transition → Feedback → Cost

再记住网络的根本现实：

Each node sees only local state.

从这一点出发：

- 距离逼出 Timer 与 Pipeline；
- 不可靠逼出 Seq、ACK 与 Retransmission；
- 共享逼出 Queue 与 Congestion；
- 规模逼出 Hierarchy 与 Routing；
- 异构逼出 Layering 与 IP；
- 没有全局即时状态，则逼出整个网络最本质的东西：**Feedback**

所以计算机网络最终不是：“TCP/IP 协议栈里有很多协议”，而是：在没有全局即时状态的世界中，大量自治节点怎样通过有限、延迟、可能不可靠的消息交换，持续修正自己的局部状态，并最终共同构造出端到端通信。这就是后续所有网络专题共同挂靠的母模型。

计算机组成原理：把 ISA 语义变成真实硬件时序

4.1 本篇的任务：计组真正研究什么

总定义

计算机组成原理研究：如何用有限数量、有限速度、有限带宽的物理硬件，正确而高效地实现 ISA 对软件承诺的程序语义。

一条指令可抽象成体系结构状态转移：

$$S_{t+1} = F_I(S_t)$$

其中 S 包含 PC、体系结构寄存器、可见内存状态、标志/控制状态等。计组所有优化的硬约束是：内部可以更复杂、更重叠，外部观察到的程序语义不能被破坏。

4.2 ISA 与微架构：What 与 How

抽象层次	定义与核心回答问题
ISA (体系结构)	• 软件可见硬件接口：指令集、寄存器、数据类型、寻址方式、异常/特权机制 • 回答“机器向软件承诺提供什么”
微架构 (Microarchitecture)	• 内部物理实现：数据通路、控制器、流水线、Cache、分支预测、内部微操作 (μops) • 回答“内部如何高效完成上述硬件承诺”

同一 ISA 可以有多种处理器实现；现代 x86 常把复杂指令解码成内部 μops ，而软件仍看到同一 ISA。RISC/CISC 是设计倾向，不是完全对立的两个盒子。

4.3 十个计组核心思想

4.3.1 指令即状态转移

指令类型虽然很多，但都可以看成对体系结构状态的合法修改：算术修改寄存器，LOAD/STORE 在寄存器与存储层次间移动数据，branch/jump 修改 PC，系统控制指令影响特权状态。

4.3.2 Data Movement：真正困难的常常是“数据在哪里”

计算 $C = A + B$ 之前必须解决：A/B 在寄存器、Cache、DRAM 还是设备？怎样寻址？走哪条总线？ALU 输入从哪个 MUX 选？结果写到哪里？因此计组大量设计是在**组织数据移动**，不是在重新发明加法。

4.3.3 Datapath / Control

Datapath 回答“有哪些路可以走”；Control 回答“此刻打开哪些路”。控制器根据 opcode、时序状态、条件标志和异常等产生寄存器写使能、ALU 操作、存储器读写、MUX 选择等信号。

4.3.4 同步状态机与关键路径

典型同步数字系统：

$$\boxed{\text{Register} \rightarrow \text{Combinational Logic} \rightarrow \text{Register}}$$

时钟周期必须容纳最坏关键路径及必要时序裕量。因此频率并非任意设定，而受最慢组合逻辑约束。

4.3.5 单周期 / 多周期 / 流水线 = 工作如何沿时间切分

单周期把整条最长指令路径塞进一个周期；多周期把工作拆开并复用硬件；流水线进一步让不同指令占据不同阶段并重叠。三者本质是在回答：**工作怎样切分，硬件怎样复用，吞吐怎样提高**。

4.3.6 Hazard = 需要的信息/资源尚未就绪

结构冒险：资源未就绪；数据冒险：值未就绪；控制冒险：下一 PC 未就绪。解决方案可统一为：**等 (stall) / 绕 (forwarding) / 加资源 / 猜 (prediction)**。

4.3.7 局部性与层次

CPU 与主存速度、容量、成本无法同时最优，于是：

$$\text{Register} \rightarrow L1 \rightarrow L2 \rightarrow L3 \rightarrow \text{DRAM} \rightarrow \text{SSD/HDD}.$$

Cache 利用时间/空间局部性，用小而快的副本隐藏大而慢的状态。

4.3.8 预测与投机

分支方向未知时，保守等待浪费流水线。预测器用历史估计未来，猜对获得性能，猜错支付 flush/recovery 成本。高性能设计不断在“平均收益 vs 错误代价”之间取舍。

4.3.9 Resource Sharing

总线、ALU、Memory port、Cache 组、DMA 通道等都是有限共享资源。结构冒险、总线仲裁、替换算法本质都是“谁在何时占用有限资源”。

4.3.10 Performance = Compute + Move + Wait

经典公式：

$$T_{CPU} = IC \times CPI \times T_{cycle} = \frac{IC \times CPI}{f}.$$

更高层看，程序时间由**有用计算**、**数据移动**、**等待停顿**组成。优化必须对准瓶颈，不能只看频率、IPC 或 MIPS 单一指标。

4.4 数据表示：有限位宽下的语义

4.4.1 整数与补码

n 位无符号范围：

$$0 \leq x \leq 2^n - 1.$$

n 位补码范围：

$$-2^{n-1} \leq x \leq 2^{n-1} - 1.$$

补码把减法统一到模 2^n 的加法框架，零只有一种编码，符号扩展自然。实际硬件减法仍需反相/进位控制和标志逻辑，不能粗暴说“CPU 完全不需要减法相关电路”。

4.4.2 ALU 与位操作

ALU 执行算术、逻辑、比较和移位；按位与/或/异或/取反用于掩码、权限字段、编码与地址处理。数据表示决定 ALU 如何解释同一位模式以及怎样判断进位、溢出和符号。

4.4.3 浮点数：有限位宽下对“范围”和“精度”的重新分配

408 中浮点数应至少掌握符号位、阶码、尾数/有效数的角色，以及 IEEE 754 单精度/双精度的基本组织思想。与定点整数相比，浮点用指数扩大动态范围，但牺牲均匀精度；规格化、舍入、溢出/下溢和特殊值都来自“有限位模式近似实数”的根本约束。做题时不要把二进制位串直接当实数，必须先按字段解释。

4.4.4 乘除、移位与溢出判断

补码加减只是数据运算的一部分。408 还常考乘法/除法的移位加减思想、算术/逻辑移位区别、无符号进位与有符号溢出的不同判定。核心仍是：**同一位模式在不同数据语义下，标志位含义不同**。

4.5 指令系统：机器能做什么

- 指令由 Opcode 与操作数/寻址信息构成；地址字段可能是寄存器号、立即数、偏移或地址计算信息，不一定是完整内存地址。
- 指令类型：算术、逻辑、移位、数据传送、访存、分支/跳转、调用/返回、系统控制。
- 高级语句一般对应多条机器指令；汇编助记符是可读表示，具体编码由 ISA 决定。

- MIPS、ARM、RISC-V 常作为 RISC 代表；x86 常作为 CISC 代表。现代实现相互吸收设计思想。

教材模型 / 工程边界：RISC-V 作为“接口/实现分离”的实例

RISC-V 官方规范明确区分 unprivileged ISA 与 privileged architecture；特权体系可以在不改变非特权 ISA 的情况下采用不同设计。这非常适合用来理解“软件可见 ISA 与 OS/硬件特权机制相互分层”。

4.6 主存组织：不仅要知道“DRAM 慢”，还要会从芯片到地址空间

408 的存储题还需要把“抽象容量”落到实际芯片组织：地址线决定可选单元数量，数据线决定每次并行读写位数；位扩展解决字长不足，字扩展解决字数不足，片选信号决定当前激活哪组芯片。多模块存储器/低位交叉编址等设计本质上是用**并行 bank**提高连续访问带宽。

SRAM/DRAM 的差异不只用于背 Cache/主存用途，还应联系刷新、速度、密度和成本。ROM/Flash 等非易失存储则用于不同持久性需求。所有这些仍可归到“容量—速度—成本—持久性”的层次权衡。

4.7 数据通路：一条指令怎样变成微操作

经典处理步骤可写成：

Fetch → Decode/RegisterRead → Execute → Memory → WriteBack.

教材中的“取指周期、间址周期、执行周期、中断周期”是指令周期的另一种划分；两套分类所处抽象层不同，不应机械混同。

关键部件：PC、IR、GPR、ALU、Control Unit、Memory Interface、Datapath。控制器不是简单“把指令翻译成电信号”，而是根据指令和机器状态生成控制选择。

4.8 单周期、多周期与流水线

4.8.1 单总线多周期

共享内部通路节省硬件，但同一时刻可进行的数据传送有限，需要把指令拆成多个微操作周期；总线容易成为资源冲突点。

4.8.2 单周期

理论 CPI=1，但时钟周期必须覆盖最慢指令完整路径，简单指令被最长路径拖慢。它适合教学与简单实现，不代表现代通用高性能 CPU 的主流微架构。

4.8.3 多周期

不同指令使用不同周期数，可复用 ALU/Memory 接口并缩短单周期关键路径。比较性能不能只看 CPI，还要结合周期时间。

4.8.4 五级流水线

IF/ID/EX/MEM/WB 只是经典模型；理想填满后 IPC 可接近 1，但单条指令仍经过多级。流水改善吞吐而不保证降低单条 latency。

三类 hazard 与常见解决：

- Structural：复制/分离硬件、stall；
- Data：forwarding、stall、编译器调度；
- Control：branch prediction、提前判定、错误路径 flush。

4.9 存储系统：Cache 不是表格题，而是副本管理

Cache 的四个统一问题：

1. Placement：主存块可以放哪里？直接/组相联/全相联；
2. Identification：如何确认当前行是不是目标块？Tag；
3. Replacement：满了牺牲谁？
4. Write：副本被修改后何时更新下一层？write-through/write-back，配合 write-allocate/no-write-allocate。

地址常分为：

Tag		Index		Offset
-----	--	-------	--	--------

分别回答“哪一个主存块 / 去哪组 / 块内哪一字节”。

SRAM 常用于 Cache，DRAM 常用于主存；寄存器、Cache、主存、辅存构成速度—容量—成本层次。多个缓存副本还会引入一致性问题，尤其在多核系统中。

4.9.1 控制器的两类经典实现

硬布线控制把控制逻辑直接做成组合/时序电路，通常速度快但修改困难；微程序控制把微操作控制序列编码进控制存储器，设计规整、扩展方便但多一层微指令解释。二者都在实现同一个映射：指令与当前状态 → 控制信号序列。

4.10 总线与 I/O：同步核心怎样接入外部世界

Bus 不是“几根线”，而是：

<i>Shared Communication Resource + Protocol</i>

需要解决寻址、数据、控制、仲裁、时序和不同速度设备协调。按传递内容可分数据/地址/控制总线；现代系统也可采用点到点互连、NoC 等，但问题模型不变。

I/O 的根本矛盾：CPU 时钟驱动且快，外设事件驱动且慢。

I/O 方式	CPU 参与度	核心思想与机制
轮询 Polling	高	• CPU 主动反复查询外设状态寄存器 • 实现简单但严重浪费 CPU 等待时间
中断 Interrupt	中	• 外设准备就绪后异步发送中断信号通知 CPU • 解除 CPU 主动轮询等待
DMA 控制	低	• CPU 仅初始化配置源/目的地址与传输长度 • DMA 控制器批量搬运数据，结束后统一中断通知

中断、异常、系统调用都可能导致控制流转移，但来源与同步性不同；CPU 需在体系结构规定的边界保存必要状态并进入处理路径。

4.11 性能：不要被单个指标骗

- Clock rate、CPI、IPC、IC 必须结合作业负载；
- $IPC \approx 1/CPI$ 只是在一致统计口径下的平均关系；乱序、多发射、空闲等会使简单直觉失真；
- 性能单位 MIPS 不适合跨 ISA/跨程序直接比较；同名 MIPS 还可能指 MIPS 指令集；
- 最可靠比较是相同任务与输入下的实际完成时间，并说明编译、功耗和硬件条件。

更高层并行方式：Superscalar、Superpipelining、VLIW、Multicore/Multiprocessor。它们受程序并行度、依赖、Cache/Memory 系统和 OS 调度限制，不能简单理解为“多放几个 CPU”。

4.12 综合题：一条 LOAD 指令的一生

以 `LOAD R1, 8(R2)` 为例：

PC→I-Cache 取指 → Decode(opcode,R1,R2,imm) → 读 R2 → ALU 计算 EA=R2+8 → D-Cache 查找 → Hit：返回数据 / Miss：逐级访问 L2/L3/DRAM → 写回 R1 → 对 ISA 呈现新的体系结构状态

如果放进流水线，还应继续问：

1. 每一步在哪个 cycle?
2. 后续指令若立即使用 R1，值在哪一级产生，forwarding 是否足够?
3. Cache miss 会造成多少 stall?
4. 地址字段如何分 Tag/Index/Offset?
5. 最终真正提交的体系结构状态是什么?

建议配合 `ADD`、`STORE`、`BRANCH` 四类母指令：纯计算、数据读入、数据写出、控制流改变，几乎覆盖教学处理器核心路径。

4.13 408 计组解题检查表

408 训练：计组六问

1. State: PC、寄存器、Cache/Memory 当前状态是什么?
2. Location: 需要的数据当前在哪里?
3. Path: 必须经过哪些部件和 MUX/总线?
4. Resource: 有没有硬件资源竞争?
5. Timing: 每个值在哪个周期产生、什么时候可用? 关键路径是什么?
6. Commit/Cost: 最终修改哪个体系结构状态, CPI/时延/命中代价是多少?

计组高频混淆

ISA $\neq$ 微架构; CPI 低 $\neq$ 程序必然快; 流水线吞吐提高 $\neq$ 单条指令变成一个周期; Cache miss $\neq$ Page Fault; 地址字段 $\neq$ 完整内存地址; 中断 $\neq$ DMA; 时钟频率高 $\neq$ 实际任务一定快; “指令和数据都是二进制” $\neq$ 所有机器都采用完全统一的物理指令/数据存储结构。

操作系统：把有限硬件变成可共享、可保护、可恢复的世界

5.1 本篇的任务：OS 五个根本任务

总定义

操作系统是运行在特权级上的、事件驱动的资源虚拟化与协调系统：它把有限而复杂的硬件资源包装成稳定抽象，让多个互不信任、彼此并发的程序安全共享资源，并在异常、I/O 和故障发生时维持可解释状态。

五个根本任务：

Control + Virtualization + Coordination + Protection + Persistence

OS 全局分析固定检查：资源与抽象是什么、谁在竞争、状态记录在哪里、什么事件改变状态、硬件与内核怎样分工、当前走哪条路径、必须保持什么不变量、代价落在哪里。

5.2 微观轴：控制权是怎样进入/离开内核的（Limited Direct Execution）

1. 根本矛盾：为什么需要 Limited Direct Execution?

CPU 只有一个核心，OS 既要让用户代码在硬件上**原生直接执行（Direct Execution）**以获得极高性能，又必须确保用户程序**不能执行特权指令、不能非法越界访存、不能永久霸占 CPU (Limited)**。因此，OS 必须依赖硬件提供的特权级与捕获机制，受控地让出与回收 CPU 控制权。

2. 对象模型：控制权转移中的核心实体

- **用户态 CPU (User Mode)**: CPL/RPL 为 3，受限与非特权指令集与 MMU 页表权限保护。
- **内核态 CPU (Kernel Mode)**: CPL/RPL 为 0，具备执行特权指令（如修改 CR3、关中断、写 IDT）的最高权限。
- **中断描述符表 (IDT)**: 系统启动时由内核初始化在内核空间，保存中断/异常号到内核处理入口点（Trap Handler）的映射。
- **用户栈 (User Stack)**: 位于用户虚拟地址空间高位/低位，保存用户函数调用的局部变量与返回地址。

- **内核栈 (Kernel Stack)**: 每个进程在内核空间独占的独立堆栈，用于保存 Trap 发生时的硬件上下文 (Trap Frame) 与内核函数调用链。

3. 状态模型：三层状态的严格划分

状态分层	所包含的具体硬件 / 数据结构信息
用户可见状态	• 用户通用寄存器（如 EAX/RAX 等）、用户栈指针 SP、程序计数器 PC • 仅由用户程序指令正常读写与修改
CPU 硬件执行状态	• 当前特权级 (Mode Bit / CPL)、页表基址寄存器 (CR3)、中断允许位 (IF) • 硬件 Trap 发生时自动关中断或切换特权级
内核管理状态	• 进程 PCB (进程状态、PID、优先级)、内核栈 (Kernel Stack)、Trap Frame • 仅由内核态代码读写，用户态完全不可见

4. 不变量 (Invariants)

1. **入口唯一性**: 用户态进入内核态的唯一合法途径是硬件 Trap 机制，绝不能通过任意 JMP/CALL 跳转到内核代码；
2. **栈隔离性**: 处于内核态执行时，必须使用当前进程的**内核栈**，绝不能为用户栈上压入内核敏感控制信息；
3. **状态可逆性**: Trap 发生时硬件与内核保存的状态，必须足以在 `return-from-trap` 时无缝恢复用户程序的执行。

5. 事件集合：触发控制权转移的三类事件

- **System Call (系统调用)**: 用户程序主动发起（如 `read()`、`fork()`），属于**同步意图事件**；
- **Exception (异常)**: 当前 CPU 执行指令内部触发（如除零、缺页 Page Fault、非法指令），属于**同步错误/缺页事件**；
- **Interrupt (中断)**: 外部硬件设备或时钟定时器异步发送（如 Timer Interrupt、Disk Ready），属于**异步外部事件**。

6. 动态轨迹：一次系统调用 (Syscall) 从用户态到内核态的单步图景

时刻	运行主体	CPU 模式	堆栈与寄存器	触发事件	硬件与内核完成的动作
T ₀	用户程序	User Mode	SP=UserStack, PC=UserApp	调用 <code>read()</code>	压参数入用户栈，加载系统调用号到指定寄存器。

T ₁	硬件 Trap	Mode Switch	硬件将 SP 切换为 KernelStack	执行 syscall / int	硬件自动保存 SS, ESP, EFLAGS, CS, EIP 到内核栈, Mode Bit 设为 0, PC 设为 IDT[trap_no]。
T ₂	Trap Entry	Kernel Mode	SP=KernelStack, 保存通用寄存器	进入 alltraps	内核汇编入口将剩余通用寄存器压入内核栈, 构造完整 TrapFrame。
T ₃	Trap Handler	Kernel Mode	SP=KernelStack, 执行 sys_read()	内核服务处理	根据系统调用号查表执行业务逻辑; 若无需阻塞, 将返回值存入 TrapFrame 中的 EAX。
T ₄	Trap Return	Kernel Mode	弹出通用寄存器, 准备恢复用户上下文	执行 sysret	从内核栈恢复 EIP, CS, EFLAGS, ESP, SS, 硬件 Mode Bit 恢复为 3 (User Mode)。
T ₅	用户程序	User Mode	SP=UserStack, PC=UserApp+4	系统调用返回	用户程序拿到 EAX 中的返回值, 继续向下执行。

7. 分支路径：控制权转移的三条演化路径

- 快路径 (直接返回): 无须等待 I/O, 更新 EAX → 执行 sysret 返回原进程用户态。
- 慢路径 (阻塞/抢占): 发起磁盘 I/O 或被时钟中断抢占 → 触发 Context Switch 切换到其他进程。
- Exception Path (严重致命): 越界/非法指令无法修复 → 向进程发送 SIGSEGV 信号或直接终止进程。

8. 机制与策略：模式切换 vs 上下文切换

408 核心概念区分：Mode Switch vs Context Switch

- Mode Switch (模式切换): 同一个进程从用户态进入内核态 (或返回)。只需在内核栈保存 CPU 寄存器上下文 (Trap Frame), 进程地址空间 (CR3)、PCB 状态不发生改变。开销小。
- Context Switch (上下文切换): CPU 从执行实体 A 切换到实体 B。内核需要保存 A 的执行上下文并恢复 B; 跨地址空间切换还要更换地址转换上下文并维护 TLB 一致性。是否把 A 置为 Ready 取决于切换原因: 抢占进入 Ready, 阻塞则进入 Blocked。

9. 408 题型映射

408 训练：控制权转移高频考点

1. **中断 vs 异常判别**：时钟中断、网卡到达属于异步中断；除零、缺页、系统调用 trap 属于同步异常。
2. **特权指令判别**：修改 CR3、关中断、写 IDT 只能在内核态执行；取指、ADD、用户态栈操作为非特权指令。
3. **栈切换考题**：特权级切换时必须切换堆栈（从用户栈切到内核栈）；同一个进程在内核态执行时使用其专属内核栈。

10. 诊断与错因矩阵

学生典型错误表象	底层认知缺口诊断	正确推演矫正句
误以为中断发生就必然发生进程切换	混淆 Mode Switch 与 Context Switch	中断处理完成后，若无更高优先级或抢占，控制权直接 <code>iret</code> 返回当前被中断进程。
认为系统调用是在用户栈上执行内核代码	未掌握用户栈与内核栈的隔离保护机制	硬件 Trap 发生的第一动作就是把 SP 换为内核栈，内核代码绝不在用户栈压栈。
误以为异常是外部硬件发出的信号	混淆 CPU 指令内部触发与外部设备中断	异常是 CPU 正在执行的当前指令同步触发的；中断是外部设备发出的异步信号。

5.3 纵轴：一个进程的一生（Life Cycle of a Process）

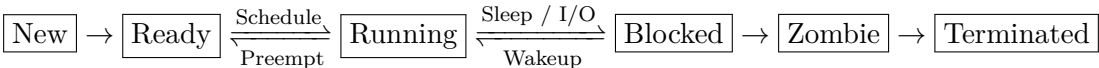
1. 根本矛盾：从静态文件到动态运行

磁盘上的程序（ELF 文件）只是静态的代码与数据字节序列。为了让程序运行，OS 必须为它建立**独立地址空间**、**分配内存**、**跟踪其执行状态**、**调度 CPU 资源**，并在其完成后清理残余资源。这一动态全生命周期管理对象即为**进程（Process）**。

2. 对象模型：进程实体在内核中的数据结构

- **PCB (Process Control Block)**: OS 管理进程的核心数据结构，包含 PID、PPID、State (Running/Ready/Blocked 等)、CPU 寄存器备份、内核栈指针、页表基址 (CR3)、打开文件描述符表指针 (FD Table) 等。
- **虚拟地址空间 (Address Space)**: 由代码段 (.text)、已初始化数据段 (.data)、未初始化数据段 (.bss)、堆 (Heap)、共享库映射区、栈 (Stack) 构成的私人地址空间。

3. 状态模型：经典进程状态机



4. 不变量 (Invariants)

- 1. **Ready 唯一缺失资源为 CPU**：就绪态进程所需的物理内存、I/O 条件均已满足，只等待调度器分配 CPU；
- 2. **Blocked 即使给 CPU 也无法运行**：阻塞态进程因等待外部事件（如磁盘 I/O 完成、信号量），即使立刻给它 CPU 也无法向前推演；
- 3. **Zombie 进程残余释放约束**：进程调用 `exit()` 后，其地址空间已销毁，但 PCB 必须保留在系统表中，直到父进程调用 `wait()` 读取其退出码后方可彻底被 OS 回收。

5. 事件集合：驱动进程状态演化的核心 API 与硬件事件

- `fork()`：克隆当前进程，创建子进程 PCB 与地址空间副本；
- `execve()`：用新 ELF 文件重构当前进程的地址空间与代码；
- `scheduler_pick()`：调度器从 Ready 队列挑出进程投入 Running；
- `timer_interrupt()`：时钟中断让内核获得记账与抢占机会；只有发生抢占时，当前实体才从 Running 转为 Ready；
- `sleep()` / `io_request()`：主动或被动等待资源，将 Running 转为 Blocked；
- `wakeup()` / `io_completion()`：外部事件完成，将 Blocked 转为 Ready；
- `exit()`：进程主动或异常终结，进入 Zombie 状态；
- `wait()`：父进程回收子进程退出状态与 PCB 空间。

6. 动态轨迹：从双击启动到进程销毁的完整生命周期表格

阶段	执行动作/API	进程状态	关键数据结构变化	触发事件	内核与硬件细节动作
1	Shell <code>fork()</code>	New → Ready	分配 PID，复制父进程 PCB、页表与文件描述符表	用户回车/启动程序	采用写时复制 (COW)，子进程与父进程共享物理页框，页表项设为 Read-Only。
2	子进程 <code>execve()</code>	Running Running	→ 替换旧程序映像，建立新代码/数据/栈映射	加载二进制程序	PID 不变；成功返回用户态后从新程序入口继续。
3	调度选中	Ready → Running	切换必要的地址空间上下文并恢复执行状态	<code>schedule()</code>	返回方式取决于体系结构与入口路径；新程序从规定入口继续执行。

4	时钟中断	Running Running Ready	→ 内核先记账并判断是否 <code>TimerInterrupt</code> 或 需要抢占	若不抢占则返回当前进程；若触发抢占，当前进程进入 <code>Ready</code> 并产生调度机会。
5	请求磁盘读	Running Blocked	→ 当前进程 <code>PCB</code> 状态写为 发起 <code>read()</code> <code>Blocked</code> ，插入 <code>Wait</code> 队列	进程放弃 <code>CPU</code> ， <code>OS</code> 启动磁盘 <code>DMA</code> 传输，调度器选择其他 <code>Ready</code> 进程运行。
6	<code>DMA</code> 完成	Blocked Ready	→ <code>PCB</code> 状态改为 <code>Ready</code> ，从 硬件 <code>Disk</code> 中断 <code>Wait</code> 队列移入 <code>Ready</code> 队列	注意：磁盘中断发生时，CPU 正在运行进程 X！ 中断程序仅将目标进程改为 <code>Ready</code> ，不保证立刻抢占！
7	进程退出	Running Zombie	→ 释放地址空间与页表，保 执行 <code>exit()</code> 留 <code>PCB</code> 与 <code>exit_code</code>	进程大部分资源已释放，但 <code>PID</code> 与 <code>exit_code</code> 留在 <code>PCB</code> 中等待父进程回收。
8	父进程回收	Zombie → Terminated	→ 从内核 <code>Task Array</code> 中彻 父进程 <code>wait()</code> 底删除 <code>PCB</code> ，回收 <code>PID</code>	父 进 程 拿 到 <code>exit_code</code> ， <code>PCB</code> 内存被彻底释放回到内核堆。

7. 分支路径：进程生命周期的三大典型分支

- 路径 A (写时复制 `COW`): `fork()` 时仅复制页表 → 任一方写入时硬件 `Page Fault` → 内核分配新物理页框。
- 路径 B (替换不换壳): `execve()` 保持 `PID` 与打开文件描述符 → 彻底覆写代码段与数据段。
- 路径 C (孤儿 vs 僵尸):
 - **Zombie (僵尸)**: 子进程先退出，父进程未调用 `wait()` → 维持 `PCB` 占用 `PID`。
 - **Orphan (孤儿)**: 父进程先退出，子进程存活 → 自动由 1 号 `init` 进程领养。

8. 机制与策略：进程调度与 `Sleep/Wakeup` 机制

408 核心机制：Blocked → Ready 与 Ready → Running 的彻底解耦

在 `OS` 中，`sleep(channel)` 与 `wakeup(channel)` 是底层原子机制：

1. **`Sleep(channel)`**: 进程主动将自己状态设为 `Blocked`，挂入 `channel` 关联的等待队列，并主动调用 `schedule()` 放弃 `CPU`。
2. **`Wakeup(channel)`**: 硬件中断（如 `Disk` 完成）触发内核唤醒函数，扫描等待队列，将阻塞在该 `channel` 上的进程状态改为 `Ready` 并移入就绪队列。
3. **关键结论**: `Wakeup` 不等于立刻拿到 `CPU` 运行！它只是获得了重新参与 `CPU` 竞聘的资格

(Ready)。当前 CPU 是否立刻切换，完全由调度策略（抢占式 vs 非抢占式、优先级）决定。

9. 408 题型映射

408 训练：进程生命周期高频考点

- 1. 进程状态转换推断：必须精准区分 Running → Ready（时间片到/抢占）、Running → Blocked（等待资源/主动 sleep）、Blocked → Ready（资源就绪/中断唤醒）。
- 2. fork() 动态返回值计算：fork() 在父进程中返回子进程 PID，在子进程中返回 0；利用循环与 fork() 考查生成进程总数（如 n 次 fork() 产生 $2^n - 1$ 个子进程）。
- 3. execve 与 PID 关系：execve 成功执行后 PID 不变，代码段与数据段更新，未设置 FD_CLOEXEC 的打开文件保持打开。

10. 诊断与错因矩阵

学生典型错误表象	底层认知缺口诊断	正确推演矫正句
以为 I/O 中断到来后，请求 I/O 的进程立刻变成 Running 运行	混淆“事件完成 (Wakeup)”与“CPU 调度 (Schedule)”	I/O 中断只负责将进程从 Blocked 转为 Ready，之后必须等待调度器 pick 才能变为 Running。
混淆 fork() 与 execve() 的功能	混淆“创建进程容器”与“重载程序内容”	fork() 创建一个与父进程一模一样的副本 (新 PID)；execve() 保持原 PID 不变，替换地址空间为新 ELF 程序。
认为进程退出后其 PCB 立即被销毁	未理解 Zombie 状态与父子进程收尸机制	进程退出后变为 Zombie，PCB 与 Exit-Code 仍保留在系统表中，只有父进程执行 wait() 后 PCB 才彻底销毁。

5.4 篇 II：并发——所有允许的交错都必须正确

并发的数学对象

并发程序不是一条执行轨迹，而是一组可能的 interleavings。正确性意味着：对于所有允许的交错，程序不变量都不能被破坏。

一条高级语句如 counter++ 可能展开为 load/add/store；上下文切换可以发生在底层步骤之间，因此“源码看起来一行”不意味着原子。

5.4.1 同步只有两类基本任务

- 1. Mutual Exclusion / Atomicity：哪些操作之间不允许其他线程插入？
- 2. Condition / Ordering：哪个条件成立后才能继续？哪个事件必须先发生？

先判断约束类型，再选 lock、condition variable、semaphore、join 等工具。

5.4.2 锁：不是布尔值，而是一个小型调度系统

高质量锁需要考虑：锁状态、原子状态转换、等待队列、唤醒、公平和性能。硬件原子指令（test-and-set、CAS、LL/SC、fetch-and-add 等）解决“原子争用”问题，但等待是否自旋、yield 还是睡眠需要 OS 支持和策略。

5.4.3 条件变量：等待的是状态谓词，不是 signal

经典模式：

```
lock(m);
while (!predicate)
    wait(cv, m);
use_state();
unlock(m);
```

条件变量只是等待队列/通知机制，真正的“条件”是共享状态上的布尔谓词。被唤醒只表示状态可能变化，因此醒来必须重新检查。

5.4.4 信号量：整数代表“许可数量”

初值 1 可表示一个互斥许可；初值 0 可表示事件尚未发生；初值 N 可表示 N 个同类资源。信号量题先问“整数的现实语义”，再写 P/V，而不是背模板。

5.4.5 并发错误与进展

非死锁错误可优先检查：

- Atomicity violation：本应整体完成的多步操作被插入；
- Order violation：程序需要 A before B，但没有机制保证。

进展性问题必须区分 Deadlock、Livelock、Starvation。死锁经典四条件是互斥、请求并保持、不可剥夺、循环等待；破坏任一必要条件可阻止死锁形成。资源分配图适合显式表示“谁持有什么、谁等待什么”。银行家算法则不是在判断“现在是否已经死锁”，而是在试探当前分配后是否仍存在一条让所有进程按最大需求完成的安全序列：Unsafe 表示未来无法保证安全，并不等于此刻已经 Deadlocked。

5.4.6 Safety / Liveness / Fairness / Performance

互斥正确并不保证公平；无死锁不保证无饥饿；正确数据结构不保证可扩展。并发性能常见根因不是“锁本身慢”，而是所有核心是否争同一个共享热点。分桶、分区、per-CPU 数据、局部累积、缩短临界区等都在降低共享。

5.4.7 经典同步模型的统一解释

生产者—消费者、读者—写者、哲学家、理发师、屏障、线程池、有界队列都应先画：

共享状态 + 容量/资源 + 顺序约束 + 互斥约束。

然后再决定同步工具。

5.4.8 事件驱动：并发不只有多线程

单线程 event loop 可按事件逐个推进状态，从而减少共享内存竞态；代价是处理函数不能长时间阻塞，需要非阻塞/异步 I/O，把长期操作拆成显式状态机、回调或 continuation。复杂性不会消失，只是从“隐式线程交错”转移成“显式状态管理”。

5.4.9 Priority Inversion：优先级与资源依赖会相互作用

高优先级线程若等待低优先级线程持有的锁，而中优先级线程不断抢占低优先级线程，就会出现优先级反转。它提醒我们：调度优先级与同步资源不能独立分析，实时系统常需要 priority inheritance 等机制控制这种依赖。

408 训练：并发题七问

1. 哪些状态共享，哪些私有？
2. 哪些源码操作其实由多步底层动作构成？
3. 必须保持什么 invariant？
4. 哪种交错会破坏它？
5. 需要互斥、顺序、资源计数还是容量限制？
6. 等待应自旋还是睡眠？
7. 是否存在 deadlock/livelock/starvation 或严重竞争？

5.5 篇 III：内存虚拟化——一块物理内存怎样像很多私人地址空间

5.5.1 核心目标：透明、效率、保护

程序看到连续虚拟地址空间；底层物理页框可以离散、共享、按需分配甚至临时不驻留。VM 首先是地址空间抽象与保护机制，不是简单“硬盘扩大内存”。

5.5.2 连续分配、分段、分页：三种不同的“空间切法”

连续分配直接给进程一段连续物理区，地址转换简单，但容易受到外部碎片和动态增长限制；分段按程序逻辑区域组织，便于保护/共享但段长可变；分页把虚拟与物理空间都切成固定大小块，用页表解除连续性要求，代价是页表、内部碎片与地址翻译开销。段页式则把逻辑分段与固定页结合。不要只背优缺点，应问：它选择固定还是可变粒度，换来了什么映射自由度和碎片类型？

5.5.3 间接映射

$$\text{Virtual Address} \rightarrow \text{Page Table/TLB} \rightarrow \text{Physical Address}$$

页面机制把地址拆为 VPN + Offset，转换只改变页号映射，页内偏移保持。多级页表用稀疏间接结构节省页表空间。

页表项不只是“地址转换”：还承载 present/valid、读写、用户/内核、dirty、accessed 等映射、保护和状态信息。

5.5.4 TLB Miss、Page Fault、Cache Miss 三分

概念	核心特征与机制区别
TLB Miss	• 地址翻译缓存未命中，页面可能已在主存中 • 可由硬件或软件页表查询填充 TLB
Page Fault	• 缺页异常，当前访问触发 OS 内核处理机制 • 包含页面未驻留 Disk I/O、COW 复制或权限越界异常
Cache Miss	• CPU 指令/数据缓存未命中 • 属于存储层次结构访问慢路径，与 TLB 独立

5.5.5 快路径与异常慢路径

$$TLB\ Hit \rightarrow TLB\ Miss + PageTable \rightarrow PageFault + I/O.$$

正常路径硬件完成，异常路径 trap 到内核，可能分配 frame、从 backing store 读页、更新 PTE/TLB，最后重启原指令。

5.5.6 页面置换与局部性

请求分页机制解决“可以按需装入”；FIFO/LRU/CLOCK/OPT 等解决“内存不够时牺牲谁”。LRU/working set 的合理性来自局部性。Thrashing 表示工作集总需求超过物理能力，大量时间浪费在换页而非有用执行。

5.5.7 延迟执行思想

Demand paging、lazy allocation、copy-on-write、write-back 都在做：先不付成本，真正需要时再付。它降低常见路径成本，但把第一次访问推入慢路径。

5.6 篇 IV：I/O——同步 CPU 怎样接入异步慢设备

I/O 的统一模型：

$$Fast\ Synchronous\ CPU \leftrightarrow Slow/Asynchronous\ Device$$

三次“卸载”：

- Polling → Interrupt：卸载等待；

- PIO → DMA：卸载搬运；
- Device-specific logic → Driver：卸载设备知识。

DMA 的典型路径：CPU 配置地址/长度 → DMA 申请总线/内存访问 → 设备与主存间批量传输 → 完成或出错中断 CPU。CPU 不是完全不参与，而是不再逐字节搬运。

5.6.1 Buffer / Cache / Spooling 不同目的

机制/抽象	核心引入目的与主要解决矛盾
Buffer	• 吸收生产者与消费者之间的速度或数据粒度差异，平滑数据流
Cache	• 保存未来可能被高频复用的数据副本，隐藏慢速存储延迟
Spooling	• 用队列与辅助存储将独占设备包装成并发共享的逻辑服务
DMA	• 批量数据传输，降低 CPU 逐字节数据搬运负担
Interrupt	• 异步事件通知，降低 CPU 主动轮询等待负担

5.6.2 磁盘调度必须先有设备成本模型

HDD 访问时间常分：

$$T_{access} = T_{seek} + T_{rotation} + T_{transfer}.$$

SSTF、SCAN 等策略的意义来自寻道/旋转成本；设备换成 SSD 后传统磁头移动模型的重要性会下降。策略必须建立在物理成本模型上。

5.6.3 经典磁盘调度策略的目标差异

FCFS 强调到达顺序与简单公平；SSTF 优先当前磁头最近请求，平均寻道可能更小但可能使远端请求饥饿；SCAN/C-SCAN 用有方向的“电梯”扫描改善全局公平与可预测性；LOOK/C-LOOK 只扫到当前最远请求。做题时先画磁道位置和服务方向，再逐步更新磁头位置，不要在脑内跳步。

5.7 篇 V：文件与持久化——从磁盘块到可命名对象

5.7.1 名称不是对象

Unix 风格文件系统的核心关系可以画成：

Name → DirectoryEntry → inode → DataBlocks

而进程访问打开文件还多一层：

$$fd \rightarrow OpenFileObject \rightarrow inode.$$

这解释了硬链接、unlink、打开引用、文件偏移等：一个 inode 可以有多个名字；删除目录项不等价于立即销毁仍被引用的对象。

5.7.2 文件系统是一组持久化映射

Name → Object → LogicalBlocks → DeviceBlocks

盘上结构典型包括 superblock、inode/data bitmap、inode table、directory data、direct/indirect blocks、data region。做题要同时看“盘上结构”与“访问方法”。

5.7.3 一次系统调用要展开成访问路径

例如：

open("/a/b/c") → pathname traversal → inode lookup → open file state.

read(fd) → fd table → file object → inode → block mapping → page cache → device.

“一个高层调用”往往对应多次元数据和数据访问。

教材模型 / 工程边界：VFS

现代 Linux 用 VFS 为用户态提供统一文件系统接口，并允许不同文件系统共存。路径查找还利用 dentry cache 加速 pathname 到 dentry/inode 的映射。408 不要求记 Linux 内核结构体，但这个实例很好地说明“统一接口 + 间接层 + 缓存”的系统思想。

5.7.4 文件访问、分配与空闲空间：三套问题不能混

- 访问方法：顺序访问、随机/直接访问等回答“应用怎样定位文件内容”；
- 分配方法：连续、链接、索引等回答“文件逻辑块怎样映射到设备块”；
- 空闲空间管理：bitmap、空闲链表等回答“哪些块还可分配”。

连续分配局部性好但扩展困难；链接分配扩展灵活但随机访问差；索引分配用额外索引结构换来随机访问与灵活扩展。inode 的 direct/indirect blocks 是索引式思想的经典实例。

5.7.5 持久化不是“调用 write 就结束”

高层更新可能对应 inode、bitmap、data、directory 等多次物理写。系统可能在任意两次稳定写之间崩溃，因此必须规定合法写顺序、提交点和恢复方式。fsck 是事后扫描修复；journaling/WAL 则先记录更新意图/内容，commit 后才能把事务视为可重做的逻辑整体。

崩溃一致性统一分析法

列出所有底层写 → 列出必须满足的先后关系 → 标记 commit point → 枚举每个 crash point → 判断忽略、redo、扫描修复或是否产生非法状态。

5.8 OS 扩展：保留系统思想，不挤占 408 主线

以下内容部分超出 408 核心考纲，用于说明持久化机制在更复杂系统中的延伸：

- RAID：在容量、可靠性、性能之间权衡；RAID0 条带无冗余，RAID1 镜像，RAID4/5 用校验，性能必须结合随机/顺序、读/写、请求大小与吞吐/延迟分析。
- FFS/Block Group：把相关 inode/目录/数据放近，体现“布局服从访问局部性”。
- LFS：把随机更新转成顺序追加，以前台快速写入换后台 cleaning；类似思想可迁移到 LSM、GC、SSD FTL。
- SSD/FTL：逻辑块 → 物理 Flash 映射；页编程、块擦除、磨损限制导致异地更新、垃圾回收和磨损均衡。
- 数据完整性：冗余负责能否重建，checksum 负责能否发现异常，backup 负责恢复历史版本；三者不能混淆。
- 分布式持久化：请求/回复可能丢失，服务器可能崩溃；幂等操作使超时重试更安全；客户端缓存提高扩展性但引入一致性语义。

5.9 Protection：不是一章，而是横向约束

CPU 的 user/kernel mode、页表权限、文件权限、特权指令、进程隔离都在回答：

Who can do What to Which Object?

每次资源访问都应检查对象是否存在、主体是否有权限、访问类型是否合法、是否越界、共享后是否需要同步。保护不是虚拟化的附属品，而是多用户/多进程系统能成立的前提。

5.10 十二个 OS 横向关系

1. 资源 抽象；
2. 时间复用 空间复用；
3. 机制 策略；
4. 状态 事件；
5. 映射 间接层；
6. 控制 特权；
7. 共享 隔离；
8. 快路径 慢路径；
9. 立即 延迟；
10. 缓存 一致性；
11. Safety Liveness Fairness；
12. Failure Recovery。

5.11 综合题一：一次 read() 的完整路径

假设用户进程执行 `read(fd, buf, 4096)`，目标数据不在页缓存：

User Mode → syscall/trap → Kernel 检查 fd 与 buf 权限 → fd→open-file→inode →
file offset→block mapping → 页缓存 miss → 发起设备 I/O → 当前进程
Running→Blocked → Scheduler 选择其他 Ready 进程并 Context Switch → DMA 搬运 →
Device Interrupt → 内核完成 I/O/更新缓存 → 原进程 Blocked→Ready → 未来再次被调度
→ 数据复制/映射给用户 → return-from-trap → User Mode

这一条路径同时串起：系统调用、特权、进程状态、调度、文件映射、缓存、I/O、DMA、中断、虚拟内存和恢复。

5.12 综合题二：一个进程的生命周期

fork/创建 → 资源与执行上下文 → exec/建立新地址空间 → demand paging/page fault →
timer interrupt/抢占 → scheduling/context switch → lock/wait/I/O blocking →
interrupt wakeup → 文件访问 → exit → 引用与资源回收

5.13 408 OS 解题检查表

408 训练：OS 九问

1. 真实稀缺资源是什么？
2. 上层操作的是哪个抽象？
3. 当前对象与状态分别是什么？
4. 什么事件触发状态变化？
5. 当前在用户态还是内核态，硬件和 OS 谁负责什么？
6. 中间有哪些映射/间接层？偏移或身份哪些保持？
7. 当前走快路径，还是进入 fault、block 或 I/O 慢路径？
8. 这是 mechanism 还是 policy？必须保持什么 invariant？
9. 最终状态和时间/空间/I/O/公平代价是什么？

OS 高频混淆

Ready ≠ Blocked；mode switch ≠ context switch；进程 ≠ 程序；线程共享地址空间 ≠ 所有状态共享；mutex/lock ≠ condition synchronization；signal ≠ condition true；unsafe state ≠ 已经 deadlocked；virtual address ≠ physical address；TLB miss ≠ page fault ≠ cache miss；buffer ≠ cache ≠ spooling；write() 返回 ≠ 数据必然已到稳定介质；文件名 ≠ inode ≠ 打开文件描述。

四科综合：从信息组织到计算系统状态推演

6.1 同一个思想在四门课里的不同化身

核心理念	数据结构 (DS)	网络 (Net)	计组 (CO)	操作系统 (OS)
抽象 / 分层	• ADT / 逻辑关系 / 存储表示	• 五层体系与协议接口	• ISA / 微架构、存储层次	• 进程、虚存、文件、VFS
状态机	• 算法维护表示与不变量	• TCP / DHCP / 路由状态	• 指令状态转移、流水线	• 进程 / 页 / 锁 / 文件状态
映射	• 下标 → 地址、关键字 → 槽位	• DNS、路由表、ARP	• 地址字段、Cache 映射	• VA→PA、fd→file→inode
局部性 / 缓存	• 连续存储、分块与索引	• DNS 缓存、网络 CDN	• Cache、TLB 概念边界	• TLB、页缓存、dentry
资源共享	• 时间、空间与预处理权衡	• 链路统计复用	• Bus / ALU / Memory 端口	• CPU / 内存 / 锁 / 设备
反馈	• 比较结果、松弛与局部修复	• ACK、超时、拥塞通知	• 分支预测、内存响应	• 中断、唤醒、缺页重试
快慢路径	• 散列直达 vs 冲突处理	• 正常发送 vs 异常重传	• Cache hit/miss、Branch hit/miss	• TLB hit vs Page Fault
保护 / 边界	• 索引、指针与合法状态边界	• 端到端作用域、协议边界	• ISA / 特权级边界	• User/Kernel、页与文件权限
性能	• 比较 / 移动 / 空间 / I/O	• 带宽 / RTT / 队列延迟	• IC / CPI / 时钟周期	• 调度 / 等待 / 缺页 I/O
故障恢复	• 回滚、重建与不变量恢复	• 丢包重传 / 路由收敛	• 异常响应 / 状态恢复	• 崩溃一致性 / 死锁恢复

6.2 数据结构怎样成为系统机制

数据结构提供可执行的组织手段，系统课程决定这些结构承载什么语义：

- 队列承载就绪进程、设备请求、网络分组和异步事件；选择 FIFO、优先队列或多级队列，就是在选择服务次序与代价；
- 树与散列表承载页表、文件索引、目录缓存、路由查找和连接状态；结构不变量决定查询速度，系统不变量决定结果能否被安全使用；
- 图模型描述网络拓扑、资源依赖和等待关系；图算法提供路径或环检测，协议与内核策略决定何时运行以及怎样处理变化；
- 位图、链表和区间结构管理页框、磁盘块与地址空间；表示选择会直接改变分配、回收与局部性成本。

因此跨科题应区分两层：数据结构解释“状态怎样组织和更新”，系统机制解释“状态代表什么、由谁拥有、何时改变”。

6.3 一条“程序读取网络数据”的系统三科路径

考虑应用执行 `recv/read(socket, ...)`：

1. 网络侧：远端数据经历应用/运输/IP/链路/物理封装与逐跳转发；
2. 网卡收到帧并通过 DMA 把数据搬到主存，产生中断；
3. 计组侧：CPU 响应中断，执行内核指令，访问 Cache/TLB/DRAM；
4. OS 侧：驱动与协议栈处理数据，唤醒阻塞在 socket 上的进程；
5. 调度器未来把 CPU 分给该进程；
6. 用户态读取返回，应用继续运行。

这说明网络、计组和 OS 不是三套世界：网络数据最终必须由真实硬件执行，并由 OS 管理资源与控制流；队列、缓冲区、索引和映射表则是组织这些状态的具体手段。

6.4 综合题解题顺序

遇到跨章节题，建议固定按照：

对象 → 层次/作用域 → 当前状态 → 事件 → 映射路径 → 资源竞争 → 状态更新 → 代价

6.4.1 第一步：先定边界，不急算

问清：现在讨论的是应用语义、OS 抽象、ISA 可见状态还是硬件内部状态？如果层次没定，公式越算越乱。

6.4.2 第二步：画动态轨迹

网络题画 packet/time line；计组画 cycle/datapath；OS 画 process/resource/state table。复杂系统题最怕只看静态定义。

6.4.3 第三步：明确“不变量”

TCP 不能把已确认之前的数据当未确认；流水线优化不能破坏 ISA 语义；并发不能破坏共享状态 invariant；文件系统恢复后必须落在允许状态集合。

6.4.4 第四步：最后才进入数字计算

位数、地址、时延、CPI、页框、P/V 值都是最后一步。先有模型，数字只是执行。

6.5 四科的训练记录模板

6.5.1 数据结构题记录

- 数据关系与存储表示；
- 操作契约与工作负载；
- 修改前后的不变量；
- 已确定区域与 frontier；
- 边界条件与进展量；
- 基本操作、规模和成本口径。

6.5.2 网络题记录

- 作用域；
- 谁保存状态；
- 触发事件；
- 字段谁读/谁改；
- 正常路径与异常路径；
- 最终性能量及单位。

6.5.3 计组题记录

- 当前体系结构状态；
- 数据位置；
- 数据通路；
- 控制与资源冲突；

- cycle/critical path;
- 最终提交状态与性能代价。

6.5.4 OS 题记录

- resource / abstraction;
- state / event;
- user/kernel 与 hardware/kernel 分工;
- mapping / data structures;
- mechanism / policy;
- invariant / wait / fault / recovery;
- 时间、空间与 I/O 代价。

完整查漏清单：防止“理解很高，但考试漏点”

7.1 数据结构覆盖清单

- 基础：抽象数据类型、逻辑/存储结构、算法特性、时间与空间复杂度、递归、最好/平均/最坏/摊还分析。
- 线性表：顺序表、单/双/循环链表、插入删除、查找、合并及边界处理。
- 栈、队列与数组：顺序/链式实现、循环队列、栈与递归、表达式与括号、矩阵地址映射、特殊矩阵压缩。
- 串：朴素匹配、KMP、前缀函数/next 的语义与推导。
- 树与二叉树：性质、存储、遍历、线索化、重建、树/森林转换、Huffman 编码、并查集。
- 查找树与优先结构：BST、AVL、红黑树概念、堆、B 树、B+ 树及插删调整。
- 图：邻接矩阵/表、DFS/BFS、连通性、生成树、Prim/Kruskal、Dijkstra/Floyd、拓扑排序、关键路径。
- 查找：顺序/折半/分块查找、判定树、散列函数、装填因子、开放定址与链地址冲突处理。
- 内部排序：插入、希尔、冒泡、快速、选择、堆、归并、基数排序；复杂度、稳定性、空间与输入敏感性。
- 外部排序：置换选择、败者树、多路归并、初始归并段与 I/O 趟数。

7.2 计算机网络覆盖清单

- 性能：rate、bandwidth、throughput、transmission/propagation/queueing/processing delay、RTT、BDP、utilization。
- OSI/TCP-IP/五层分析模型的对应关系、PDU、encapsulation/decapsulation。
- 应用：DNS、HTTP/版本、WWW、C/S、P2P、FTP、SMTP/POP3/IMAP。
- 运输：port/socket、mux/demux、TCP 4-tuple/5-tuple、UDP、TCP 字节流、握手/关闭/TIME_WAIT、Seq/ACK、累计确认、快速重传、SACK、滑窗、flow/congestion control。
- 网络：IP 尽力而为服务、转发与路由、网关、IPv4 首部与分片、CIDR/子网/聚合/最长前缀匹配、DHCP、ICMP、NAT/NAPT、IPv6、距离向量/链路状态、RIP/OSPF/BGP、自治系统、多播、移

动 IP。

- 链路：framing、transparent transmission、HDLC stuffing、parity/CRC/Hamming、PPP、MAC、多路复用、ALOHA/CSMA、CSMA/CD、CSMA/CA、MAC address、ARP/NDP、hub/bridge/switch、self-learning/flooding、collision/broadcast domain、VLAN。
- 物理：source/sink/signal/channel、bit/symbol rate、encoding/modulation、Nyquist/Shannon/SNR、media、repeater/hub、circuit/message/packet switching、datagram/virtual circuit。

7.3 计算机组成原理覆盖清单

- 系统概述与性能：clock、IC、CPI、IPC、MIPS、CPU time；
- 指令/ISA：opcode、operand/address fields、machine instruction、assembly/mnemonic、instruction types、RISC/CISC、x86/MIPS/ARM/RISC-V；
- 数据表示与运算：bit width、unsigned/signed、sign-magnitude/ones' complement/two's complement/biased、modular arithmetic、定点乘除与移位、overflow、IEEE 754 浮点、ALU；
- 存储：register/cache/DRAM/SRAM/ROM/secondary storage、memory chip 位/字扩展、bank/interleaving、hierarchy、locality、mapping/replacement/write policy、coherence 概念；
- 冯·诺依曼与 Harvard/modified Harvard 边界；
- 指令周期与微操作：取指、间址、执行、中断周期，以及 IF、ID、EX、MEM、WB 阶段；PC、IR、通用寄存器、ALU、控制器和数据通路；
- hardwired vs microprogrammed control；
- single-bus、single-cycle、multi-cycle、pipeline；三类 hazard 与 forwarding/stall/prediction；
- bus：data/address/control、arbitration/timing/synchronization；
- I/O：polling、interrupt、DMA、设备异步性；
- superscalar、superpipelining、VLIW、multicore/multiprocessor。

7.4 操作系统覆盖清单

- 总纲：resource/abstraction、time/space multiplexing、mechanism/policy、state/event、indirection/mapping、快路径/慢路径、locality/cache、lazy/demand、isolation/protection。
- 控制与特权：user/kernel、system call、trap/exception/interrupt、mode/context switch、timer。
- Process/Thread：状态、PCB/执行上下文概念、创建/终止/阻塞/唤醒、thread shared/private state、IPC（pipe/shared memory/message/notification）。
- Scheduling：FCFS/SJF/SRTF/RR/Priority 等经典模型，指标与 starvation/fairness。
- Concurrency：race/critical section/atomicity/order、hardware atomic primitives、locks、condition variables、semaphores、经典同步模型、event-driven model、deadlock four conditions、banker/safe state、livelock/starvation/priority inversion、lock granularity/scalability。

- Memory: contiguous allocation、segmentation/paging/segment-paging、fragmentation、page table/multi-level、TLB、VM/demand paging、page fault、replacement FIFO/LRU/CLOCK/OPT、working set/thrashing、COW/lazy allocation、protection/sharing。
- I/O: device interface/driver、polling/interrupt/DMA、buffer/cache/spooling、disk physical cost、disk scheduling。
- File: file/directory/path/fd/open-file/inode、hard/symbolic link、access methods、contiguous/linked/indexed allocation、bitmap/free-list、direct/indirect mapping、path lookup、VFS 思想。
- Persistence extension: RAID、FFS locality、journaling/crash consistency、LFS、SSD/FTL、checksum vs redundancy vs backup、distributed retry/idempotence/cache consistency。

参考资料与边界说明

8.1 资料层次

1. 考试层：以 408 常见教材模型、题设和计算规则为作答依据；
2. 机制层：参考 OSTEP 与 MIT xv6，解释虚拟化、并发、持久化以及 trap、页表和文件系统怎样运行；
3. 工程边界：使用 RFC、RISC-V 规范和 Linux Kernel Documentation 限制对 HTTP/3、IPv6、特权架构、VFS 与调度器的绝对化表述。

8.2 推荐外部资料

- Operating Systems: Three Easy Pieces: pages.cs.wisc.edu/~remzi/OSTEP/
- MIT xv6 / 6.1810: pdos.csail.mit.edu/6.S081/2024/xv6.html
- RISC-V Ratified Specifications: docs.riscv.org/reference/home/index.html
- RFC 9000 QUIC: rfc-editor.org/rfc/rfc9000.html
- RFC 9114 HTTP/3: rfc-editor.org/rfc/rfc9114.html
- RFC 8200 IPv6: rfc-editor.org/rfc/rfc8200.html
- Linux Kernel Documentation: docs.kernel.org/

8.3 最后的学习原则

不要背现象，要寻找生成现象的机制

数据结构不要只背代码，要知道关系怎样表示、操作维护什么不变量、成本为何产生；网络不要只背协议字段，要知道作用域、状态、反馈和资源竞争；计组不要只背框图，要知道数据在哪里、经过什么路径、在哪个时钟边界提交；OS 不要只背状态和算法，要知道真实资源、抽象、事件、机制、策略和必须维持的不变量。

最终目标不是“见过这道题”，而是“没见过，也能从关系、约束、不变量与代价推出可行路径”。